

COMPTE RENDU DE TP

TPR202 – Virtualisation & Conteneurisation

TP1 – VirtualBox | TP2 – VMware ESXi | TP3 – Docker

IUT de Béthune – Département Réseaux & Télécommunications
Année universitaire 2025-2026

TP1 – VirtualBox

Objectif : Prendre en main l'hyperviseur de type 2 VirtualBox, créer et configurer une machine virtuelle Ubuntu, et explorer ses fonctionnalités avancées.

1. Travail préliminaire – Création de la machine virtuelle

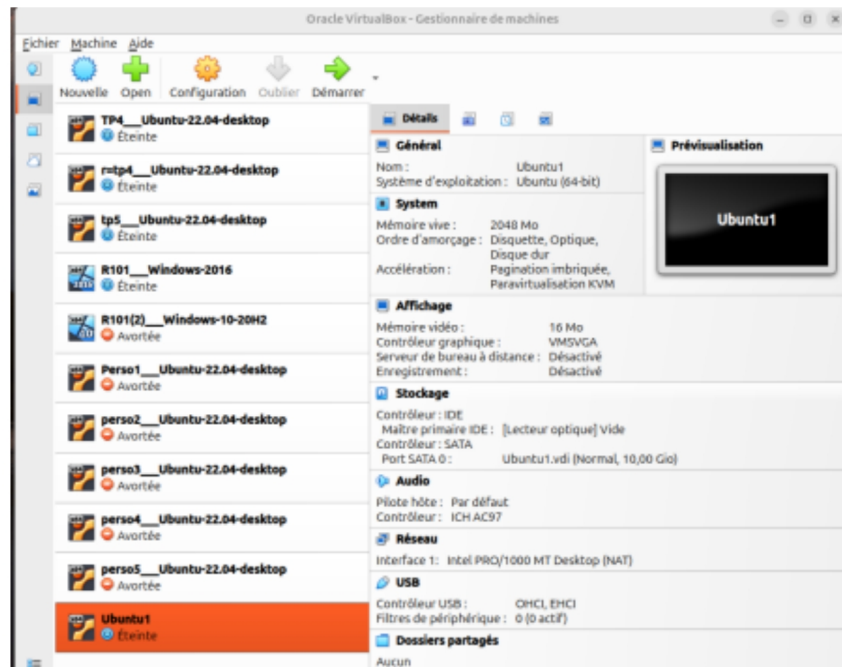
Démarche

Création d'une VM via l'interface graphique de VirtualBox avec les paramètres : Nom Ubuntu1, Ubuntu 64 bits, 10 Go de stockage dynamique.

Compte rendu

Travail préliminaire

Création de la machine virtuelle depuis VirtualBox :



VirtualBox – Gestionnaire de machines avec la VM Ubuntu1 créée

Analyse des fichiers générés

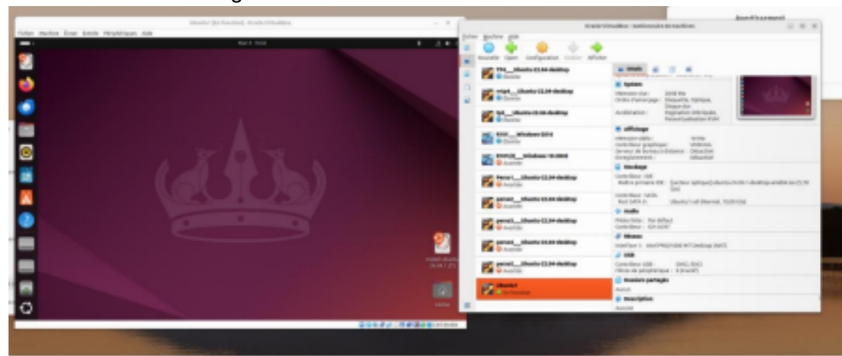
- .vbox → Fichier XML de configuration (RAM, CPU, réseau, chemins des disques virtuels)
- .vdi → Disque dur virtuel dynamique (taille réelle << 10 Go au départ)
- Logs/ → Fichiers journaux contenant l'historique de démarrage, erreurs matérielles virtuelles

```
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$ ls
libuntui.vbox  libuntui.vbox-prev  libuntui.vdi
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$ file libuntui.vdi
libuntui.vdi: VirtualBox Disk Image, major 1, minor 1 (aka Oracle VirtualBox Disk Image), 18737418240 bytes
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$ file libuntui.vbox
libuntui.vbox: XML 1.0 document, ASCII text
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$ file libuntui.vbox-prev
libuntui.vbox-prev: XML 1.0 document, ASCII text
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$ ls -lh libuntui.vdi
-rw-r--r-- 1 ayman_ettarni etudiant 2,8M mars  4 15:02 libuntui.vdi
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$ ls -lh libuntui.vbox
-rw-r--r-- 1 ayman_ettarni etudiant 2,4K mars  4 15:02 libuntui.vbox
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$ ls -lh libuntui.vbox-prev
-rw-r--r-- 1 ayman_ettarni etudiant 2,4K mars  4 15:02 libuntui.vbox-prev
ayman_ettarni@j001-14:/srv/vhoms/ayman_ettarni/VirtualBox_VMs/Ubuntu1$
```

Commande file – identification des types de fichiers .vbox, .vdi et .vbox-prev

Installation et démarrage de la VM

Montage de l'ISO Ubuntu en lecteur CD/DVD virtuel et démarrage de l'installation :



Installation et démarrage de la VM Ubuntu depuis l'ISO

Conclusion

Le disque dynamique .vdi croît progressivement selon les données écrites, économisant l'espace hôte. Les fichiers .vbox et logs permettent à VirtualBox de reconstruire et journaliser l'état complet de la VM.

2. Les Additions invité

Démarche

Installation des paquets nécessaires depuis la VM :

```
sudo apt-get install build-essential linux-headers-$(uname -r)
```

```
ubuntu@ubuntu:~$ nano /etc/apt/apt.conf.d/proxy
ubuntu@ubuntu:~$ sudo nano /etc/apt/apt.conf.d/proxy
ubuntu@ubuntu:~$ sudo apt-get install build-essential linux-headers-$(uname -r)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
linux-headers-6.8.0-41-generic is already the newest version (6.8.0-41.41).
linux-headers-6.8.0-41-generic set to manually installed.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu dpkg-dev fakeroot g++
  g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libbinutils
  libbct1-0 libctf-nobfd0 libctf0 libdpkg-perl libfakeroot
  libfile-fcntllock-perl libgcc-13-dev libgprofng0 libhwspan0 libitm1 liblsan0
  libquadmath0 libstdc++-13-dev libubsan2 libubsan1
  lto-disabled-list make
Suggested packages:
  binutils-doc gprofng-gui debconf-keyring g++-multilib g++-13-multilib
  gcc-13-doc gcc-multilib autoconf automake libtool flex bison gcc-doc
  gcc-13-multilib gcc-13-locales gdb-x86-64-linux-gnu git bzr libstdc++-13-doc
  make-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential dpkg-dev
  fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libbinutils
  libbct1-0 libctf-nobfd0 libctf0 libdpkg-perl libfakeroot
  libfile-fcntllock-perl libgcc-13-dev libgprofng0 libhwspan0 libitm1 liblsan0
  libquadmath0 libstdc++-13-dev libubsan2 libubsan1
  lto-disabled-list make
```

Installation de build-essential et linux-headers dans la VM

Exécution du script d'installation des Additions invité :

```
sudo ./VBoxLinuxAdditions.run
```

```

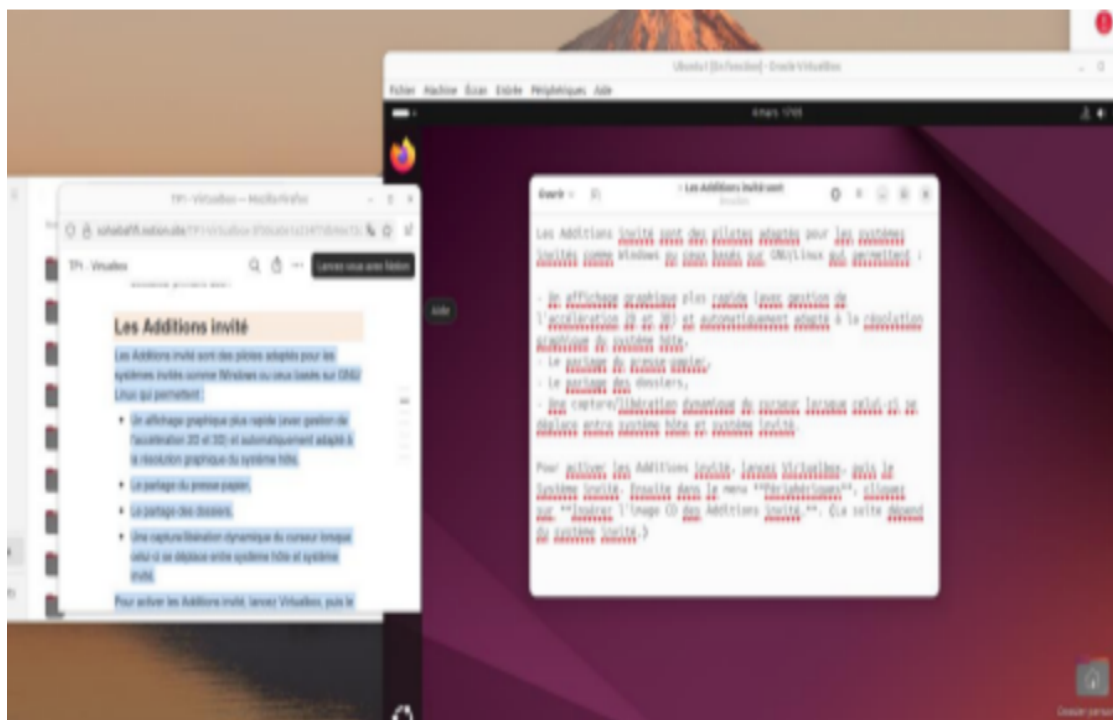
0 upgraded, 37 newly installed, 0 to remove and 0 not upgraded.
Need to get 8 8156 7 MB of archives.
Execution de la commande d'installation :
ayman@ayman-VirtualBox: /media/ayman/VBox_GAs_7.2.6$ sudo ./VBoxLinuxAdditions.run
Paramétrage de g++-13 (13.3.0-6ubuntu2-24.04.1) ...
Paramétrage de g++ (4:13.2.0-7ubuntu1) ...
update-alternatives: utilisation de « /usr/bin/g++ » pour fournir « /usr/bin/c++ » (c++) en mode automatique
Paramétrage de build-essential (12.10ubuntu1) ...
Traitement des actions différées (« triggers ») pour man-db (2.12.0-4build2) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.39-0ubuntu0.3) ...
ayman@ayman-VirtualBox: /media/ayman/VBox_GAs_7.2.6$ sudo ./VBoxLinuxAdditions.run
Verifying archive integrity... 100% MD5 checksums are OK. All good.
Uncompressing VirtualBox 7.2.6 Guest Additions for Linux 100%
VirtualBox Guest Additions: Starting.
VirtualBox Guest Additions: Setting up modules
VirtualBox Guest Additions: Building the VirtualBox Guest Additions kernel
modules. This may take a while.
VirtualBox Guest Additions: To build modules for other installed kernels, run
VirtualBox Guest Additions: /sbin/rcvboxadd quicksetup <version>
VirtualBox Guest Additions: or
VirtualBox Guest Additions: /sbin/rcvboxadd quicksetup all
VirtualBox Guest Additions: Building the modules for kernel 6.8.0-41-generic.
update-initramfs: Generating /boot/initrd.img-6.8.0-41-generic
VirtualBox Guest Additions: Running kernel modules will not be replaced until
the system is restarted or "rcvboxadd reload" triggered
VirtualBox Guest Additions: reloading kernel modules and services
VirtualBox Guest Additions: kernel modules and services 7.2.6 r172322 reloaded
VirtualBox Guest Additions: NOTE: you may still consider to re-login if some
user session specific services (Shared Clipboard, Drag and Drop, Seamless or
Guest Screen Resize) were not restarted automatically
ayman@ayman-VirtualBox: /media/ayman/VBox_GAs_7.2.6$

```

Exécution de VBoxLinuxAdditions.run – installation des modules kernel

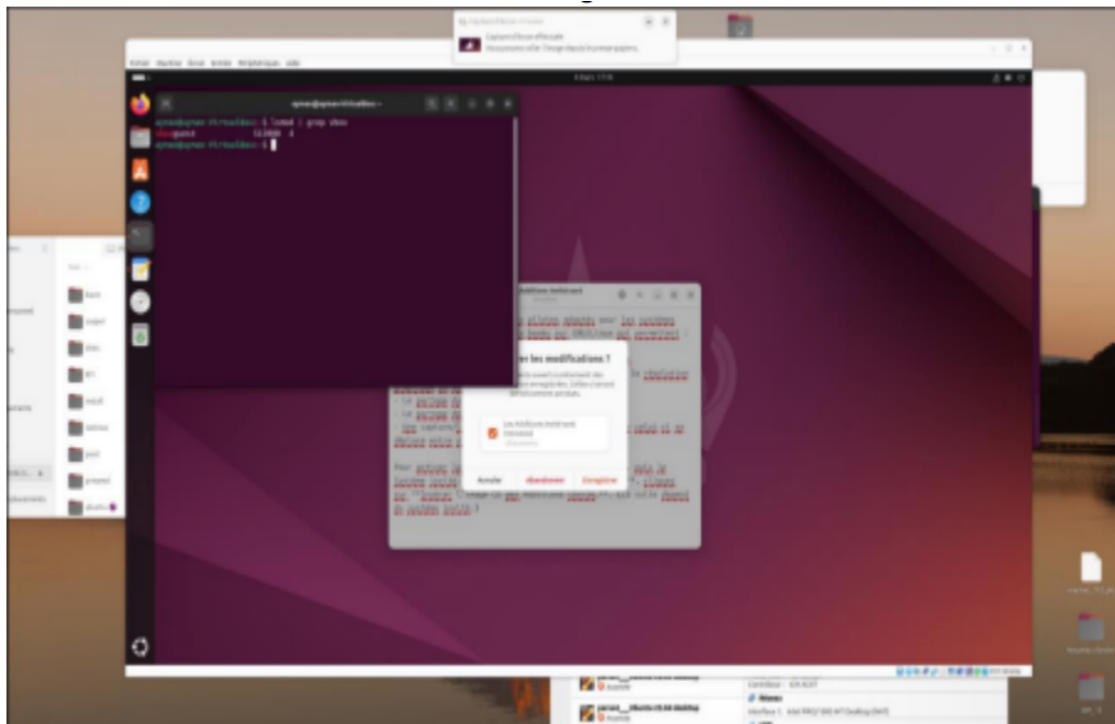
Fonctionnalités testées

Test du partage de presse-papier (copier-coller entre hôte et invité) :



Test du partage de presse-papier bidirectionnel hôte ↔ invité

Test du redimensionnement automatique de l'affichage :



Redimensionnement automatique de la résolution d'écran

Conclusion

Les Additions invité améliorent considérablement l'expérience utilisateur : résolution graphique adaptative, presse-papier partagé et capture/libération transparente du curseur.

3. Configuration réseau

Mode NAT – Vérification connectivité et adresse IP

Vérification de l'accès Internet et observation de l'adresse IP en mode NAT (ip addr show + ping) :

```

Fichier Machine Écran Entrée Périphériques Aide
4 mars 17:26
ayman@ayman-VirtualBox: ~
--- 172.31.25.9 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.727/0.999/1.439/0.265 ms
ayman@ayman-VirtualBox: ~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp8s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:08:0e:97 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp8s3
        valid_lft 8633sec preferred_lft 8633sec
    inet6 fd17:625c:f037:2:5d16:aaad:8824:47aa/64 scope global temporary dynamic
        valid_lft 8633sec preferred_lft 14333sec
    inet6 fd17:625c:f037:2:080:27ff:fe88:ea97/64 scope global dynamic mngtnpaddr
        valid_lft 8633sec preferred_lft 14333sec
    inet6 fe80::a00:27ff:fe88:e97/64 scope link
        valid_lft forever preferred_lft forever
ayman@ayman-VirtualBox: ~$ ping -c 4 172.31.25.9
PING 172.31.25.9 (172.31.25.9) 56(84) bytes of data:
64 bytes from 172.31.25.9: icmp_seq=1 ttl=255 time=0.691 ms
64 bytes from 172.31.25.9: icmp_seq=2 ttl=255 time=0.419 ms
64 bytes from 172.31.25.9: icmp_seq=3 ttl=255 time=0.557 ms
64 bytes from 172.31.25.9: icmp_seq=4 ttl=255 time=0.593 ms

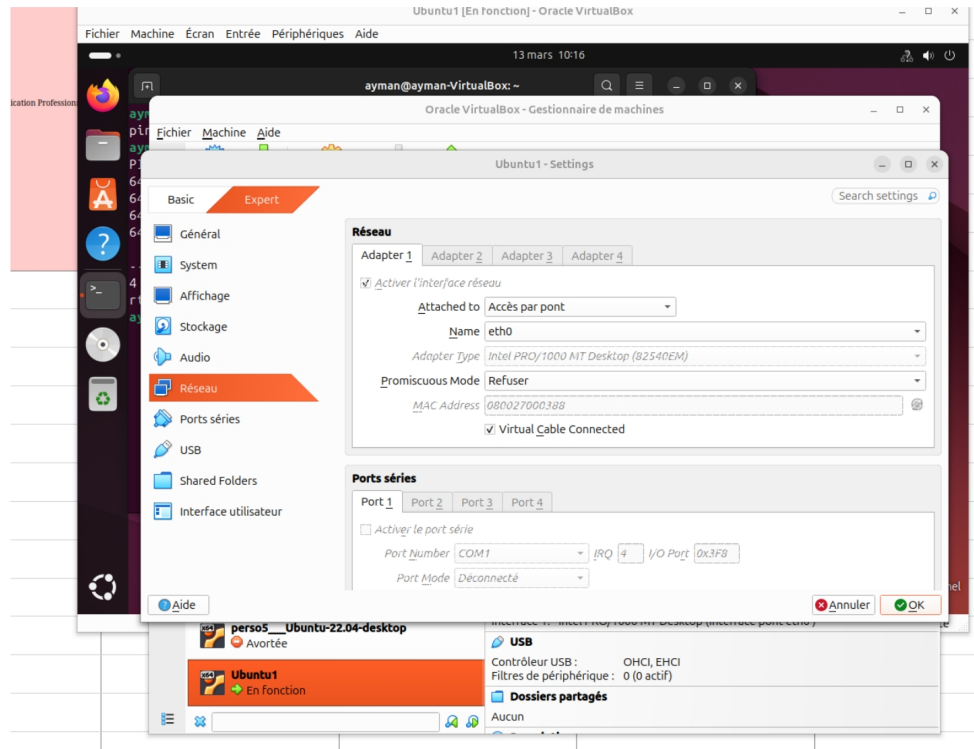
--- 172.31.25.9 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.419/0.565/0.691/0.097 ms
ayman@ayman-VirtualBox: ~$

```

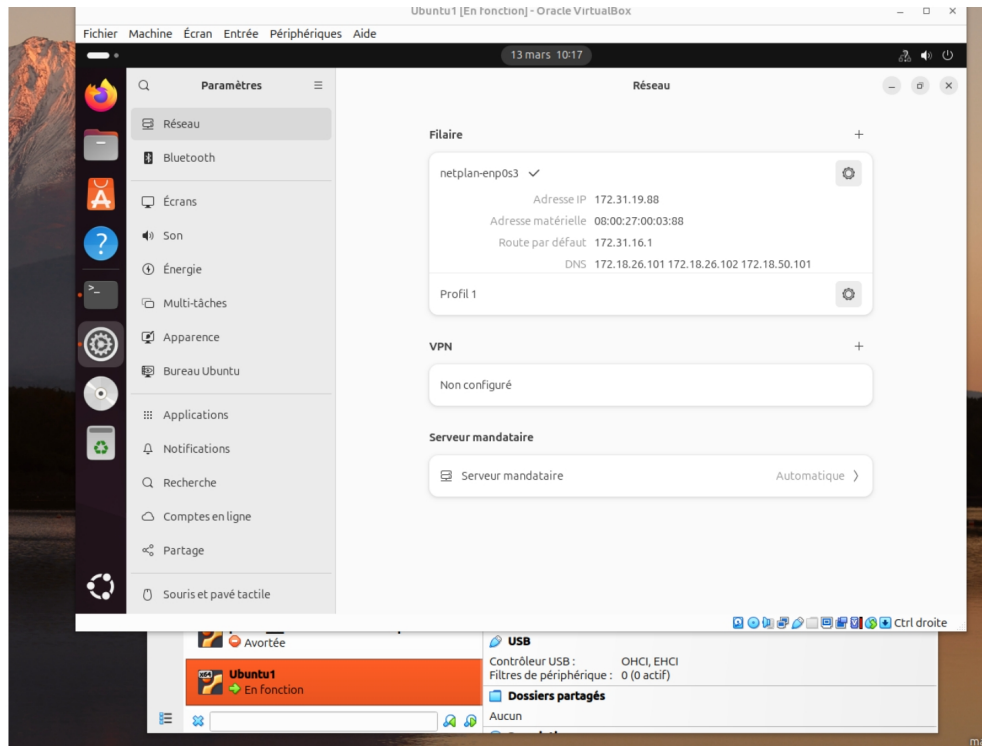
Mode NAT – adresse IP 10.0.x.x et vérification de la connectivité Internet

Mode Bridge

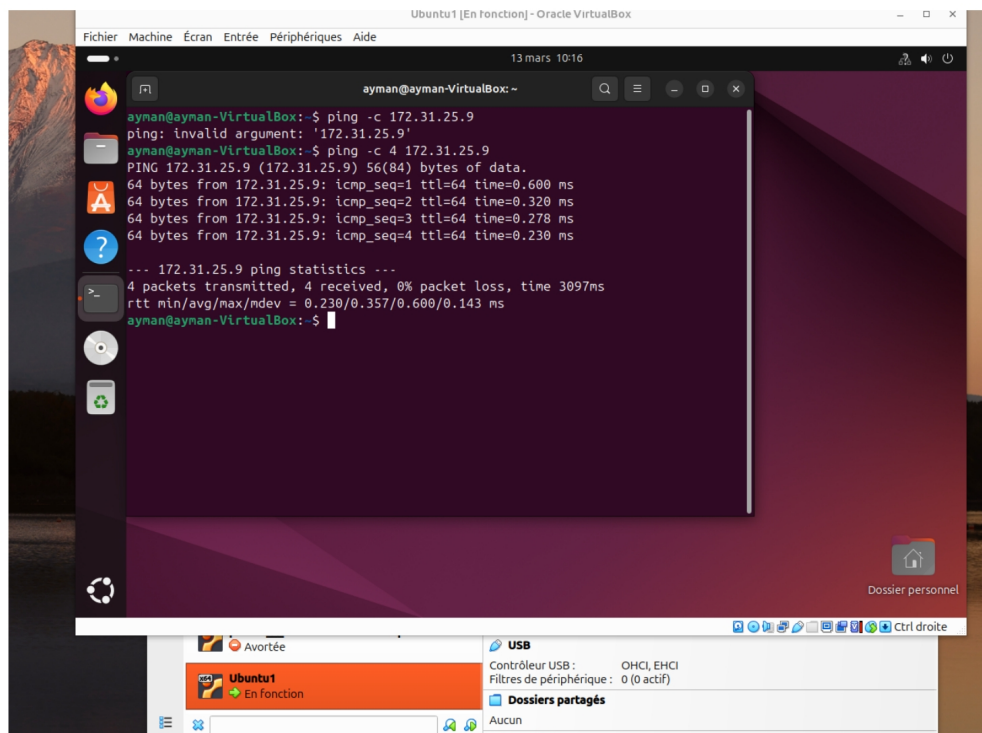
Configuration du mode Accès par pont. La VM obtient une IP du réseau local via DHCP :



Configuration du mode Bridge – paramètres réseau VirtualBox



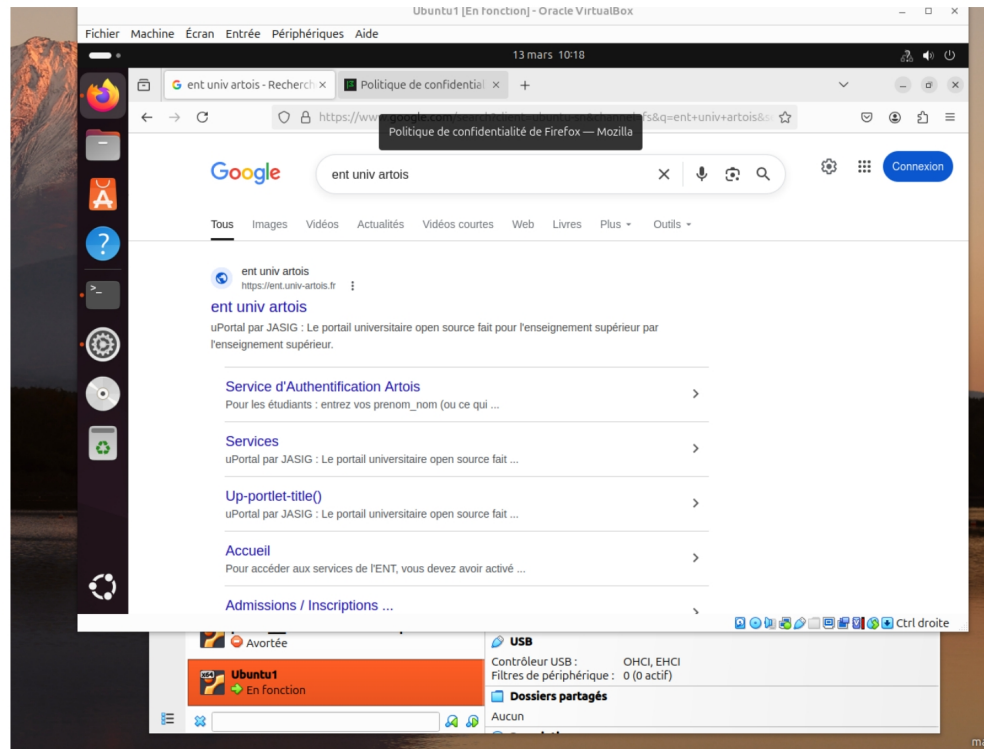
VM obtenant l'adresse IP 172.31.19.88 via DHCP en mode Bridge



Test de connectivité – ping 172.31.25.9 depuis la VM en mode Bridge

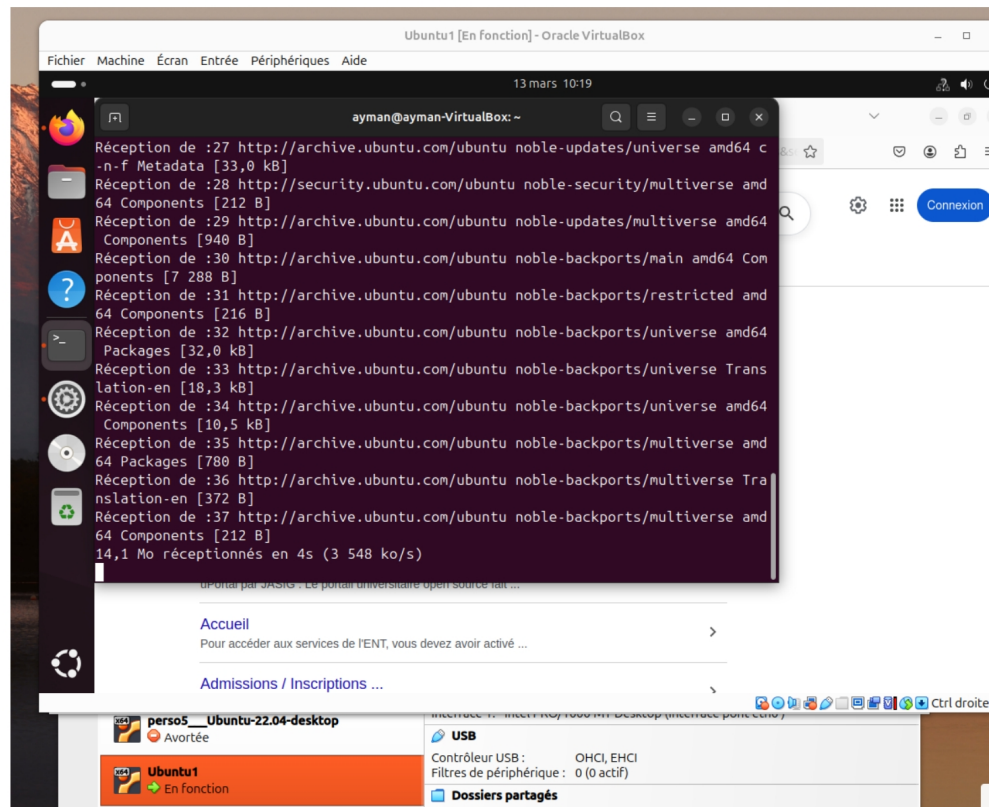
Exercice pratique – Installation SSH et connexion distante

Vérification de l'accès Internet depuis la VM puis installation du serveur SSH :



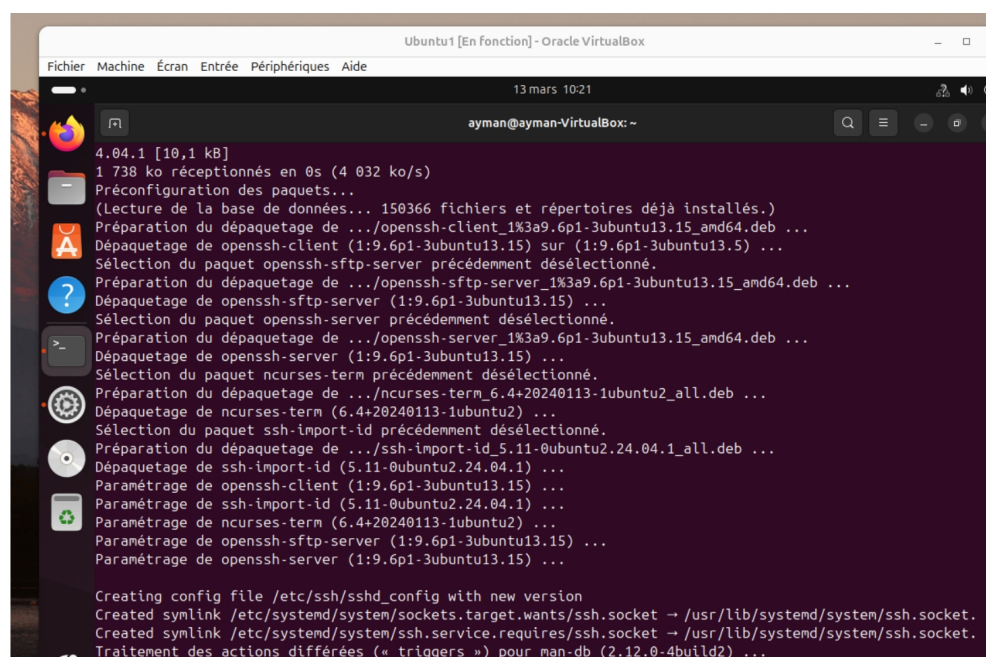
Vérification de l'accès Internet depuis la VM (navigation Firefox)

```
sudo apt-get update
```



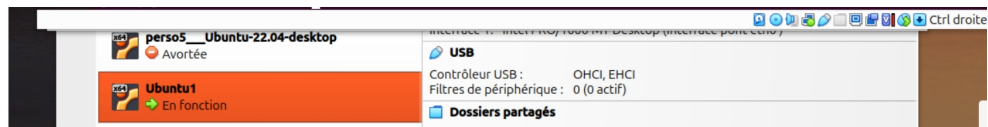
sudo apt-get update – mise à jour des dépôts

sudo apt-get install openssh-server

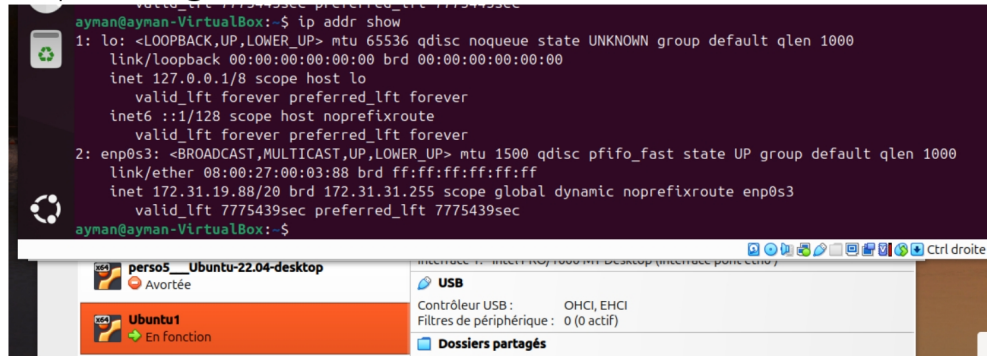


Installation de openssh-server dans la VM

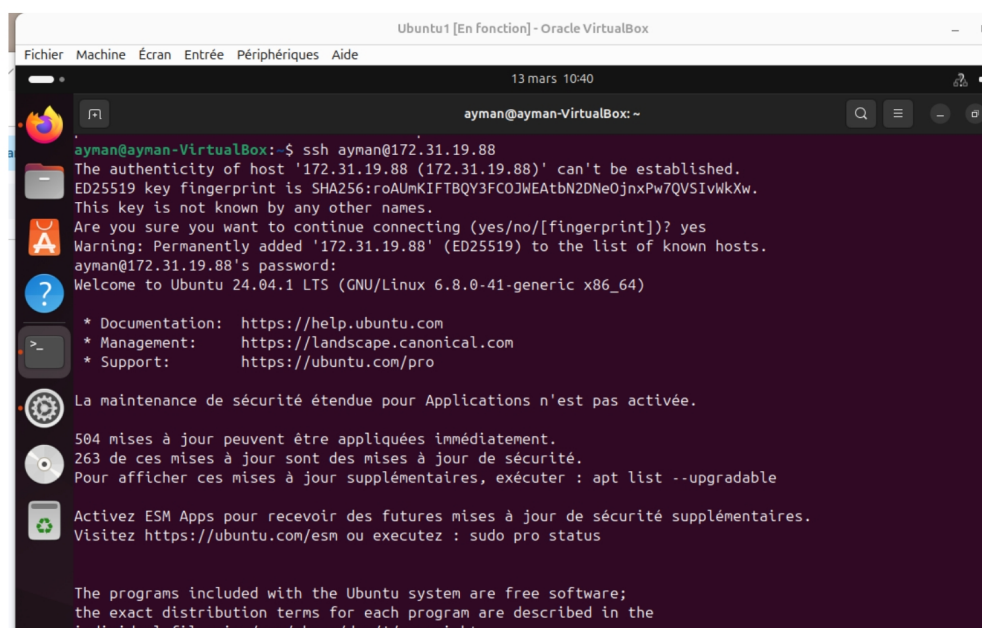
Récupération de l'adresse IP de la VM et connexion SSH depuis la machine hôte :



Récupération de l'IP:



ip addr show – adresse IP 172.31.19.88 de la VM



Connexion SSH réussie vers la VM (ssh ayman@172.31.19.88)


```

individual files in /usr/share/doc/ /copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ayman_ettarni@j009-06:~$ ping 172.31.19.88
PING 172.31.19.88 (172.31.19.88) 56(84) bytes of data.
64 bytes from 172.31.19.88: icmp_seq=1 ttl=64 time=0.197 ms
64 bytes from 172.31.19.88: icmp_seq=2 ttl=64 time=0.148 ms
64 bytes from 172.31.19.88: icmp_seq=3 ttl=64 time=0.126 ms
64 bytes from 172.31.19.88: icmp_seq=4 ttl=64 time=0.141 ms
^C
--- 172.31.19.88 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3085ms
rtt min/avg/max/mdev = 0.126/0.153/0.197/0.026 ms
ayman_ettarni@j009-06:~$

```

Ping depuis la machine hôte vers la VM – communication bidirectionnelle confirmée

Conclusion

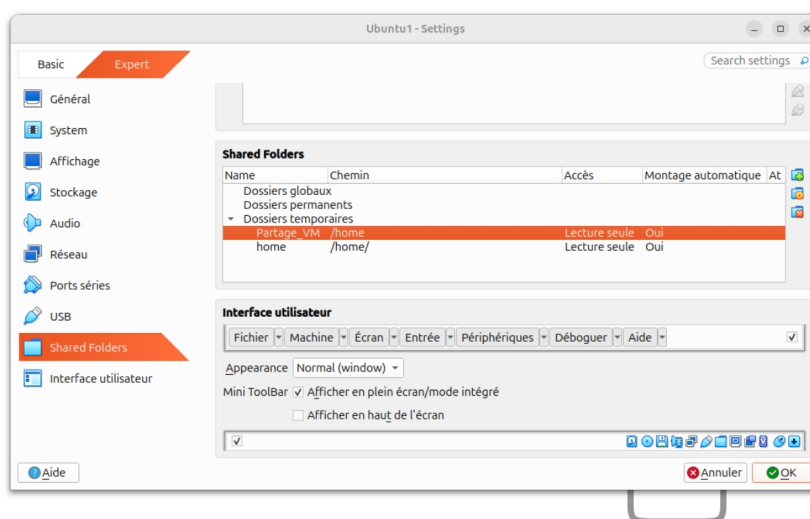
Le mode NAT isole la VM derrière l'hôte, le mode Bridge l'intègre au réseau local. SSH permet une administration distante sécurisée de la VM depuis n'importe quelle machine du réseau.

4. Partage de dossier entre hôte et invité

Démarche

Configuration du dossier partagé dans VirtualBox (Paramètres > Dossiers partagés). Ajout de l'utilisateur au groupe vboxsf :

```
sudo adduser user vboxsf
```



Configuration du dossier partagé Partage_VM dans VirtualBox – montage automatique activé

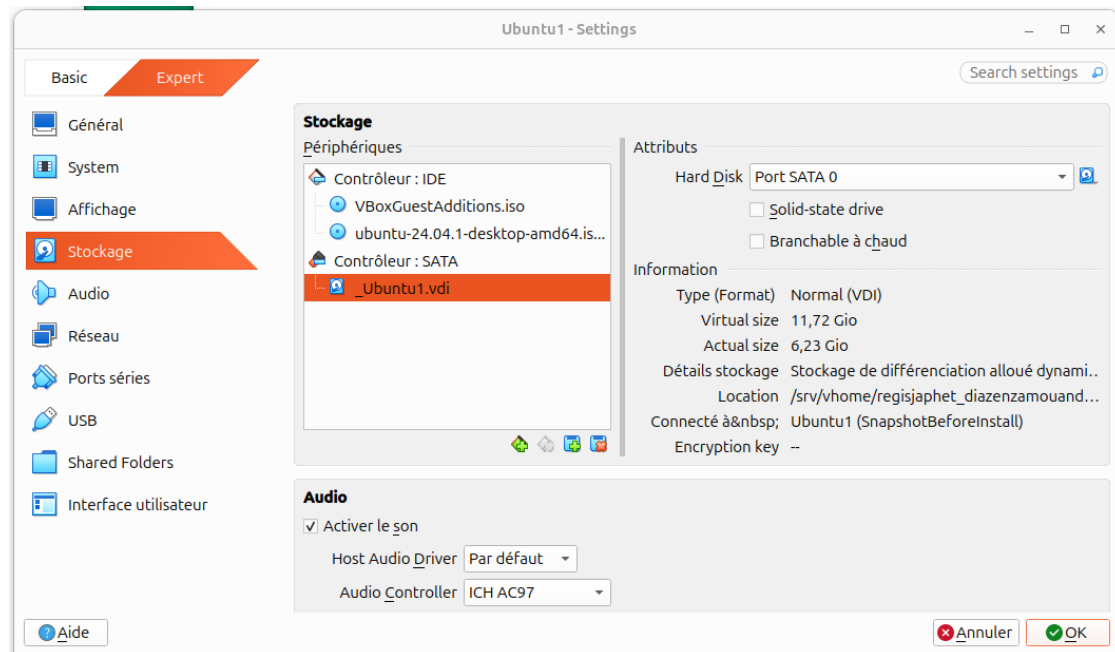
Conclusion

Le partage de dossiers via vboxsf permet un échange transparent de fichiers. L'appartenance au groupe vboxsf est nécessaire pour que l'utilisateur standard puisse y accéder en lecture/écriture.

5. Gestion des snapshots

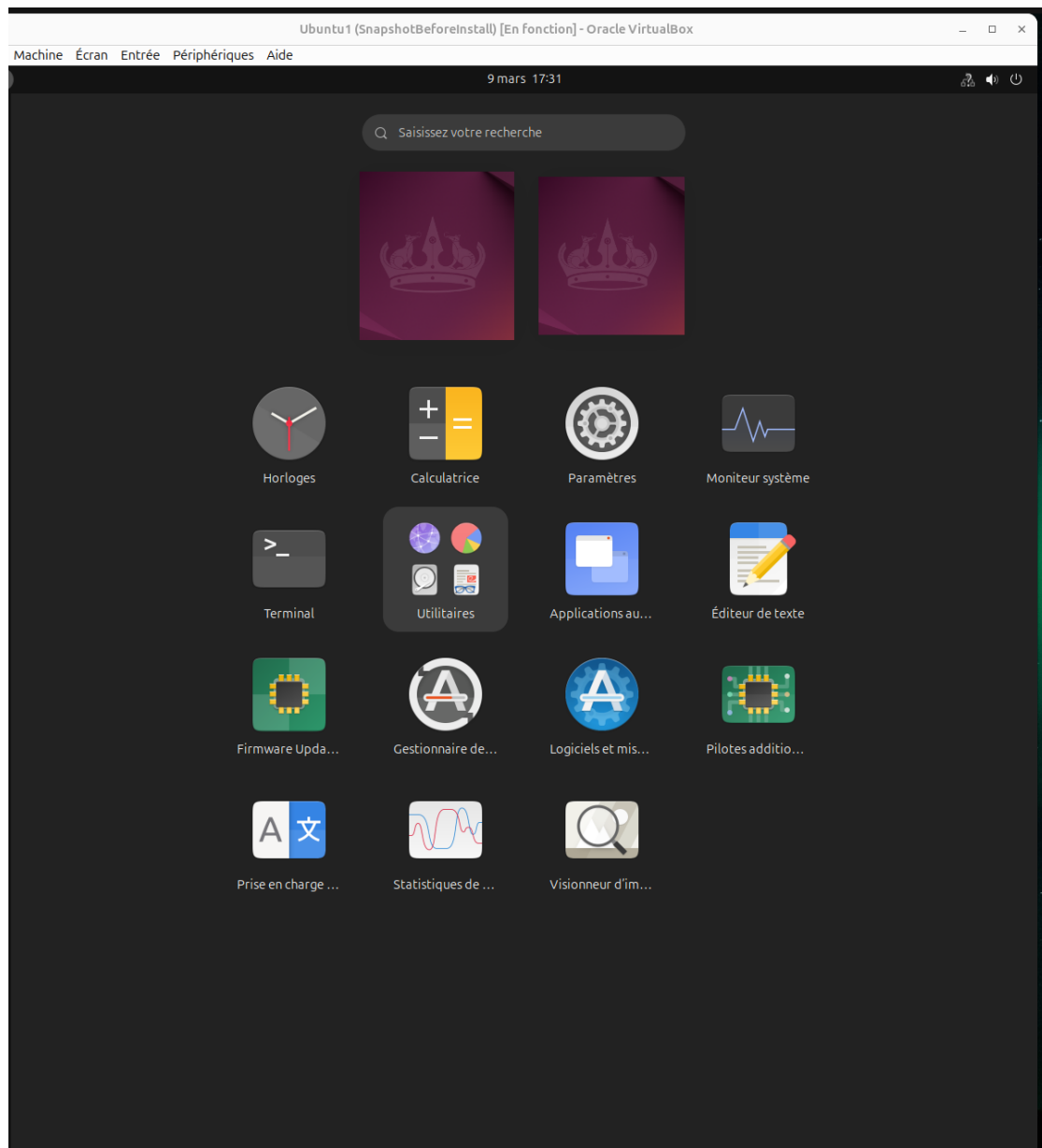
Démarche

Création d'un snapshot avant modification, puis restauration pour annuler tous les changements :



Création d'un snapshot – état initial de la VM sauvegardé

Installation d'un logiciel dans la VM et modification de la configuration :



Installation d'un logiciel dans la VM après le snapshot

Restauration du snapshot – les logiciels installés sont supprimés, la VM revient à l'état initial :

```

user@user-VirtualBox: /media/user/VBox_GAs_7.2.6$ sudo apt-get install build-essential linux-headers-$(uname -r)
[sudo] Mot de passe de user :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
linux-headers-6.8.0-41-generic est déjà la version la plus récente (6.8.0-41.41).
linux-headers-6.8.0-41-generic passé en « installé manuellement ».
Les paquets supplémentaires suivants seront installés :
  binutils binutils-common binutils-x86-64-linux-gnu bzip2 dpkg-dev fakeroot
  g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libbinutils
  libcc1-0 libctf-nobfd0 libctf0 libdpkg-perl libfakeroot
  libfile-fcntllock-perl libgcc-13-dev libgprofng0 libhwasan0 libitm1 liblsan0
  libquadmath0 libsframe1 libstdc++-13-dev libtsan2 libubsan1
  lto-disabled-list make
Paquets suggérés :
  binutils-doc gprofng-gui bzip2-doc debian-keyring g++-multilib
  g++-13-multilib gcc-13-doc gcc-multilib autoconf automake libtool flex bison
  gcc-doc gcc-13-multilib gcc-13-locales gdb-x86-64-linux-gnu git bzip
  libstdc++-13-doc make-doc
Les NOUVEAUX paquets suivants seront installés :
  binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2
  dpkg-dev fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu
  gcc gcc-13 gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu
  libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl
  libasan8 libbinutils libcc1-0 libctf-nobfd0 libctf0 libdpkg-perl libfakeroot
  libfile-fcntllock-perl libgcc-13-dev libgprofng0 libhwasan0 libitm1 liblsan0
  libquadmath0 libsframe1 libstdc++-13-dev libtsan2 libubsan1
  lto-disabled-list make
0 mis à jour, 38 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 56,2 Mo dans les archives.
Après cette opération, 208 Mo d'espace disque supplémentaires seront utilisés.

```

Restauration du snapshot – retour à l'état avant modification

Conclusion

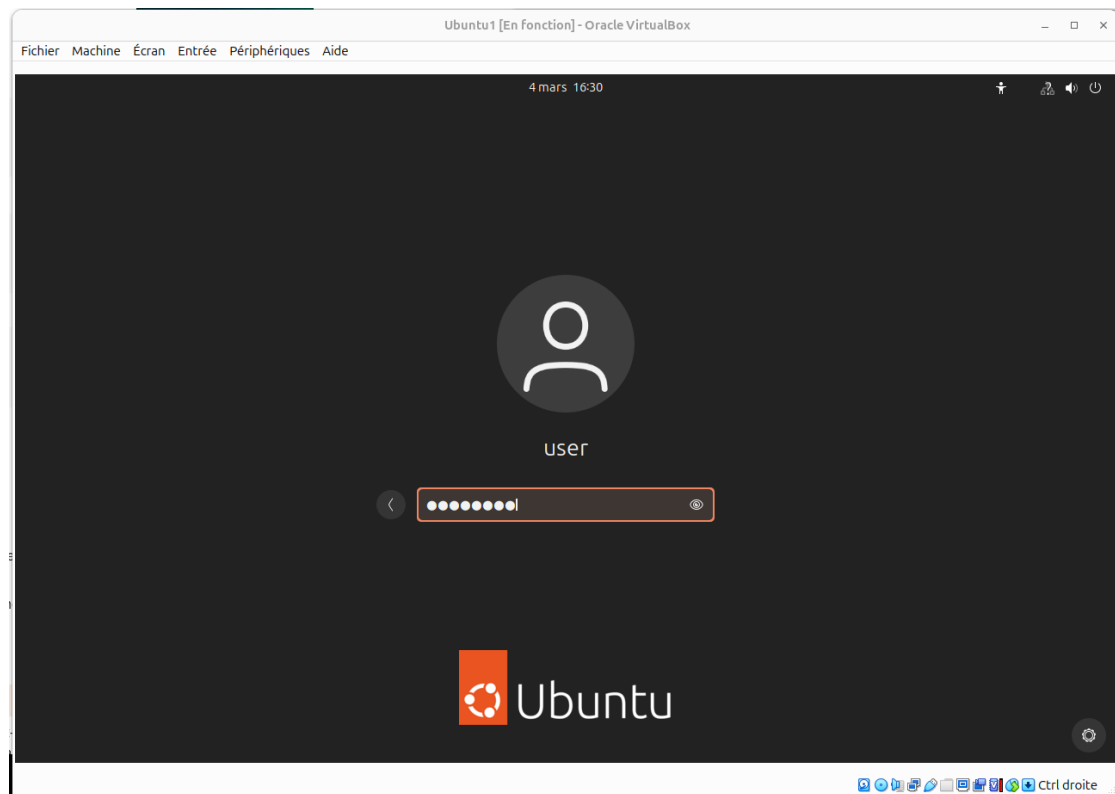
Les snapshots permettent d'expérimenter sans risque. Après restauration, tous les logiciels installés et modifications effectuées depuis le snapshot sont annulés. Outil indispensable pour les environnements de test.

6. Gestion et format des disques virtuels

Redimensionnement d'un disque VDI

Vérification de la taille du .vdi avant redimensionnement, puis agrandissement avec VBoxManage :

```
VBoxManage modifymedium disk /chemin/vers/disk.vdi --resize
<nouvelle_taille_en_Mio>
```



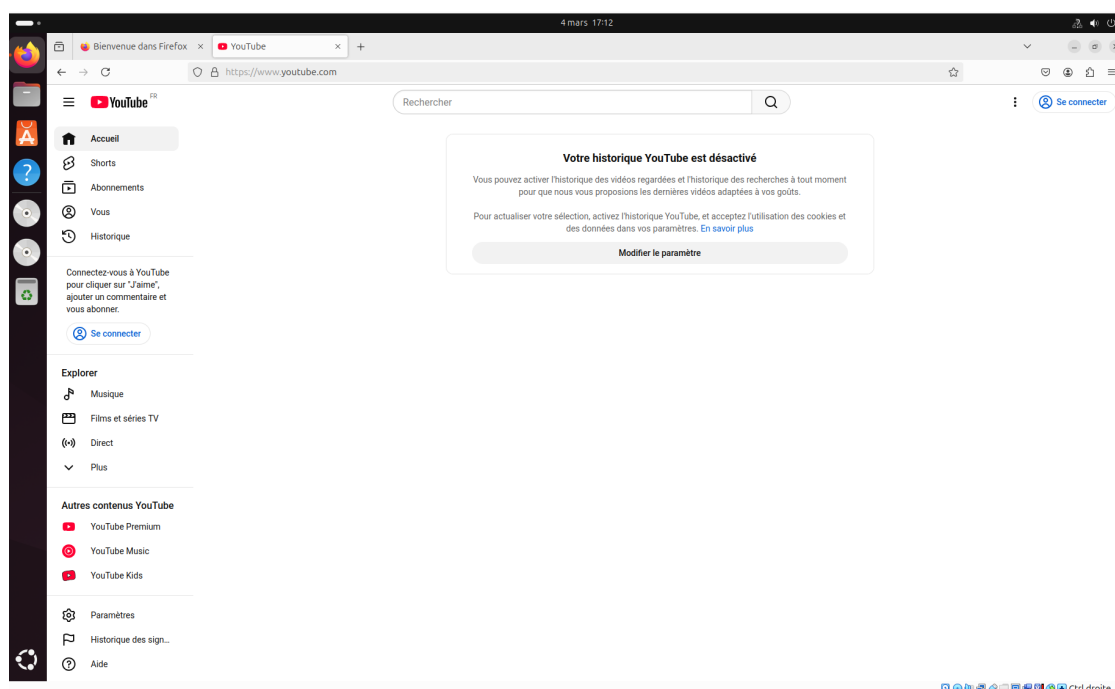
Taille initiale du fichier .vdi – disque dynamique

Exécution de la commande de redimensionnement – taille passée de 10 Go à 11,72 Go :

```
regisjaphet_diazenzamuanda@j001-08:~$ VBoxManage createvm --name "UbuntuCLI" --register
Virtual machine 'UbuntuCLI' is created and registered.
UUID: 16d9c008-b87c-4c3b-9d0c-194f0ec4cd4f
Settings file: '/srv/vhome/regisjaphet_diazenzamuanda/VirtualBox_VMs/UbuntuCLI/UbuntuCLI.vbox'
regisjaphet_diazenzamuanda@j001-08:~$ VBoxManage modifyvm "UbuntuCLI" --memory 2048 --ostype "Ubuntu_64"
regisjaphet_diazenzamuanda@j001-08:~$ VBoxManage createhd --filename UbuntuCLI.vdi --size 10000
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Medium created. UUID: d8933dfa-e75c-466f-bdb0-8ab7b7ef4525
regisjaphet_diazenzamuanda@j001-08:~$ VBoxManage storagectl "UbuntuCLI" --name "SATA Controller" --add sata --controller IntelAHCI
regisjaphet_diazenzamuanda@j001-08:~$ VBoxManage storageattach "UbuntuCLI" --storagectl "SATA Controller" --port 0 \
> --device 0 --type hdd --medium UbuntuCLI.vdi
regisjaphet_diazenzamuanda@j001-08:~$
```

VBoxManage modifymedium – redimensionnement du disque virtuel

Agrandissement de la partition dans la VM avec gparted :



gparted – agrandissement de la partition après resize du .vdi

Conclusion

Le redimensionnement d'un disque VDI se fait en deux étapes : agrandir le fichier avec VBoxManage, puis étendre la partition dans la VM avec gparted. Le disque dynamique occupe seulement l'espace réellement utilisé.

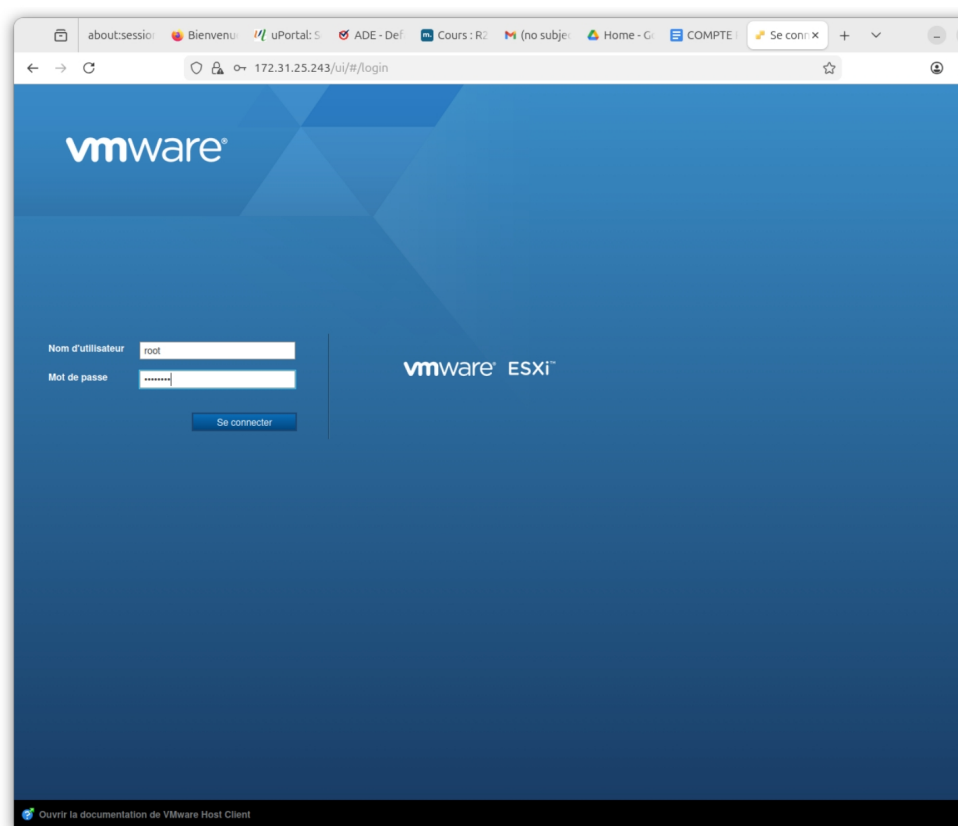
TP2 – VMware ESXi

Objectif : Découvrir un hyperviseur de type 1 (bare-metal), créer une VM sur un serveur ESXi distant et faire communiquer des VMs entre elles.

1. Accès et observation de l'interface ESXi

Démarche

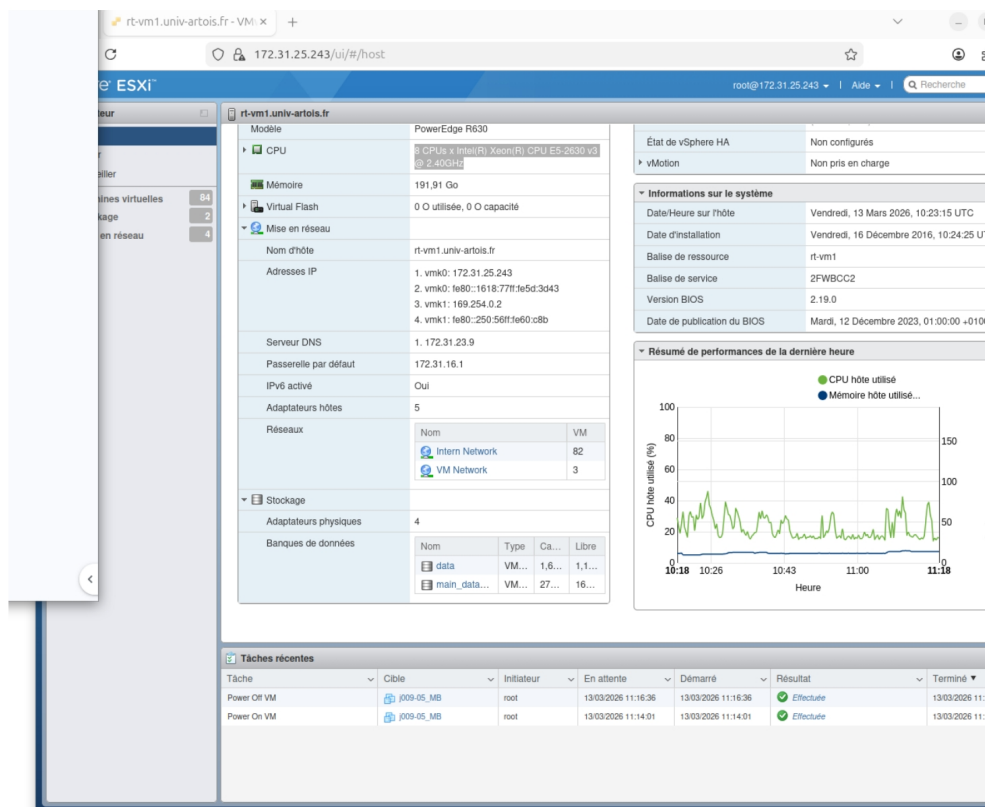
Accès à l'interface web ESXi via <http://172.31.25.243> (rt-vm1.univ-artois.fr). Identifiants : root / progt00



CPU utilisé : 8 CPUs x Intel(R) Xeon(R) CPU E5-2630 v3 @ 2.40GHz

Page de connexion VMware ESXi – interface web

Observation des performances du serveur hôte : CPU 8x Intel Xeon E5-2630 v3 @ 2.40 GHz, mémoire 191,91 Go :



Dashboard ESXi – performances CPU/mémoire, réseau, stockage et tâches récentes

La courbe verte représente le pourcentage de processeur hôte utilisé, la ligne bleue la mémoire hôte utilisée.

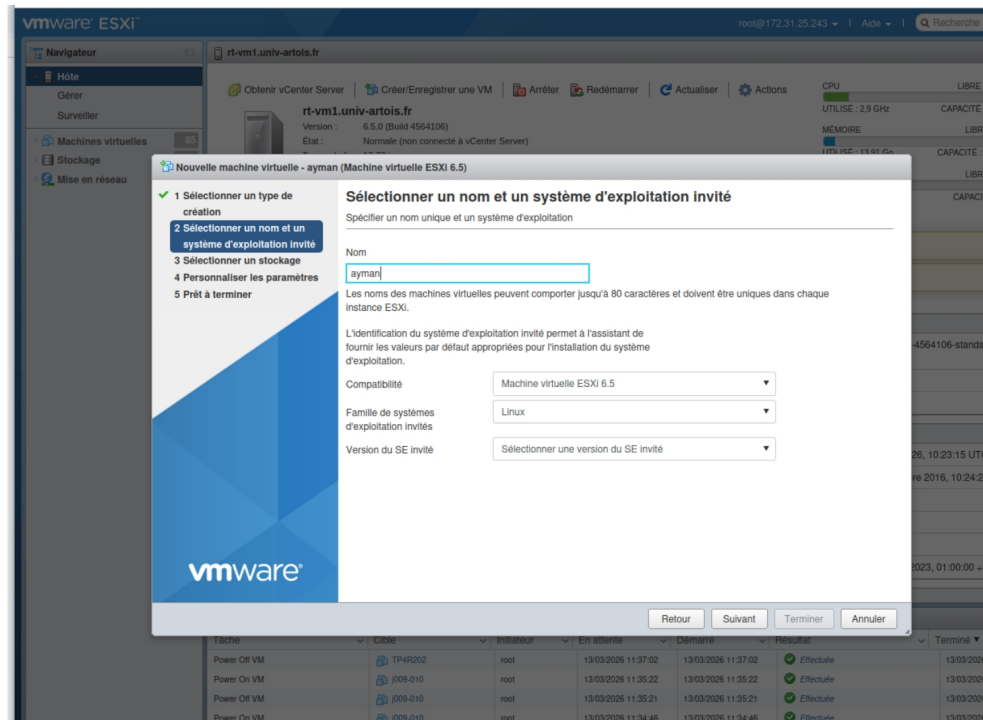
Conclusion

ESXi s'exécute directement sur le matériel (type 1). Son interface web centralise la gestion des VMs, du réseau et du stockage. Les métriques en temps réel reflètent les ressources physiques du serveur (PowerEdge R630).

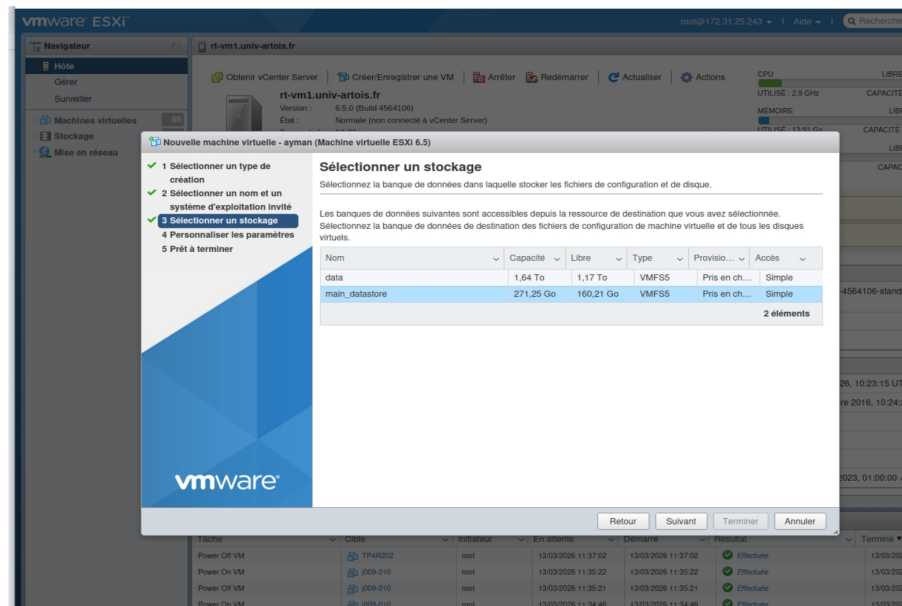
2. Création et installation de la VM

Démarche

Création d'une VM (Ubuntu Linux 64 bits, 10 Go, réseau Intern Network, ISO depuis /nas/isos) :

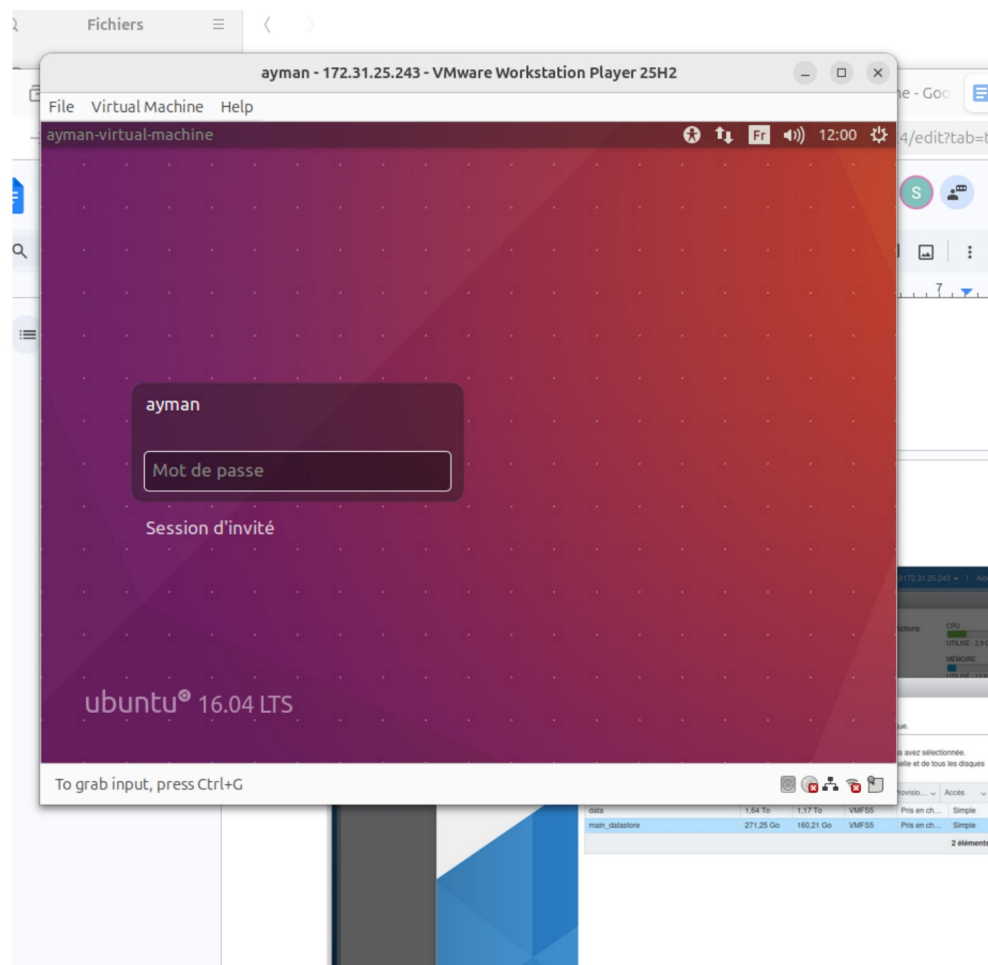


Assistant de création VM ESXi – sélection du nom et du système d'exploitation

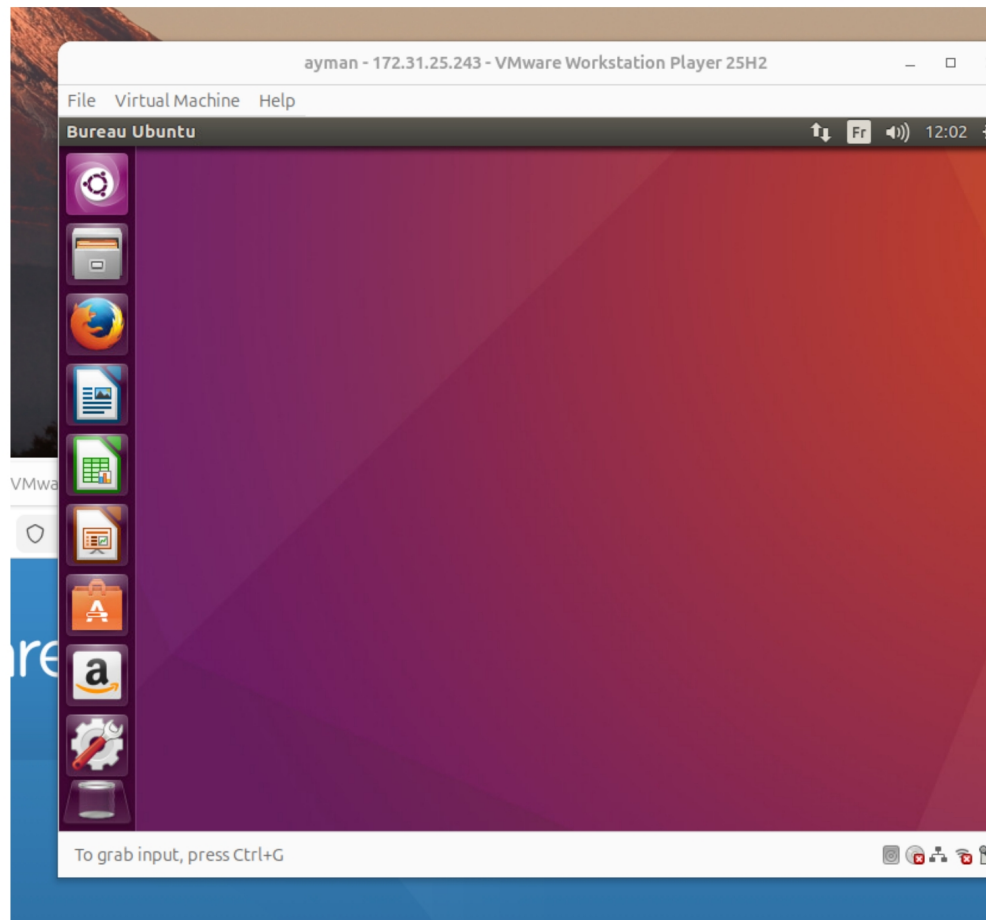


Sélection du stockage – banques de données main_datastore (271 Go)

Démarrage de la VM et installation d'Ubuntu :



VM Ubuntu démarrée sur ESXi – écran de connexion Ubuntu 16.04 LTS



Bureau Ubuntu opérationnel dans la VM ESXi

Observation des performances après installation

Après démarrage de la VM, le processeur est plus sollicité et la mémoire hôte utilisée augmente.

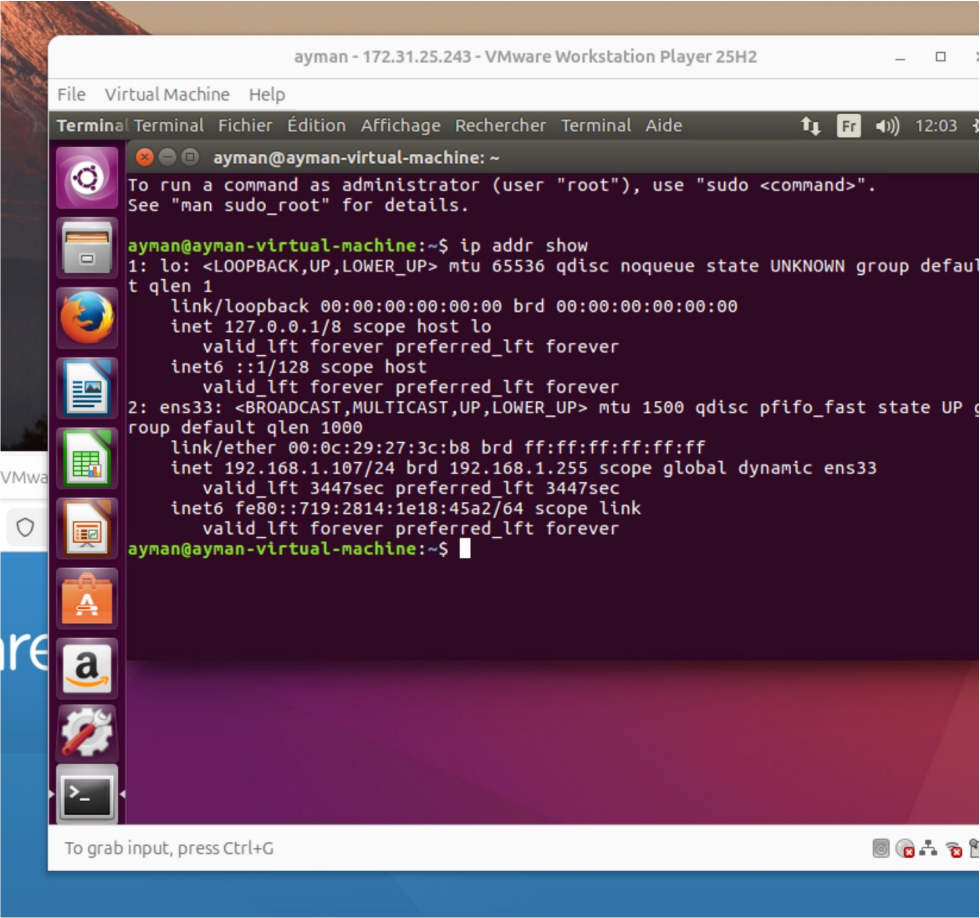
Conclusion

La création d'une VM sur ESXi s'effectue via l'interface web en quelques étapes. L'impact sur le serveur physique est visible en temps réel, illustrant le partage de ressources typique d'un hyperviseur de type 1.

3. Communication entre VMs

Démarche

Vérification de l'adresse IP de la VM, puis tests de connectivité (ping) vers la VM d'un collègue :

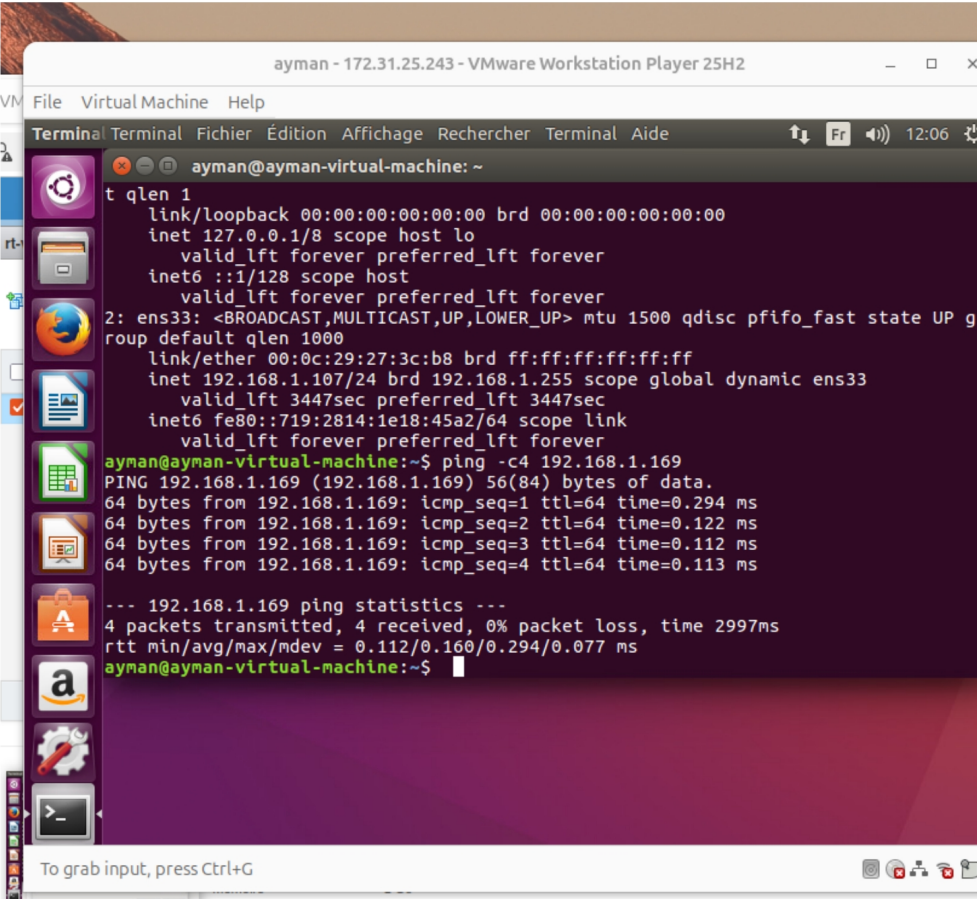


The screenshot shows a VMware Workstation Player window titled 'ayman - 172.31.25.243 - VMware Workstation Player 25H2'. The terminal window inside the VM shows the following commands and output:

```
ayman@ayman-virtual-machine: ~  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ayman@ayman-virtual-machine:~$ ip addr show  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP g  
    link/ether 00:0c:29:27:3c:b8 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.1.107/24 brd 192.168.1.255 scope global dynamic ens33  
        valid_lft 3447sec preferred_lft 3447sec  
    inet6 fe80::719:2814:1e18:45a2/64 scope link  
        valid_lft forever preferred_lft forever  
ayman@ayman-virtual-machine:~$
```

Ping vers une autre vm d'un collègue :

ip addr show – adresse IP 192.168.1.107 de la VM sur ESXi



The screenshot shows a VMware Workstation Player window titled 'ayman - 172.31.25.243 - VMware Workstation Player 25H2'. The terminal window is open, displaying the output of the 'ifconfig' command for the 'ayman' virtual machine. The output shows the loopback interface 'lo' and the ethernet interface 'ens33'. The 'ens33' interface is configured with IP address 192.168.1.107 and a netmask of 24. The terminal also shows the output of a 'ping' command from the 'ayman' VM to the IP address 192.168.1.169, which is another VM on the same network. The ping is successful, showing 4 packets transmitted, 4 received, and 0% packet loss. The terminal window has a menu bar with 'File', 'Virtual Machine', and 'Help'. The terminal window also has a title bar with 'Terminal', 'Terminal', 'Fichier', 'Édition', 'Affichage', 'Rechercher', 'Terminal', and 'Aide'. The terminal window is running on a Linux desktop environment with a purple background and a sidebar with application icons.

```
ayman@ayman-virtual-machine: ~  
t qlen 1  
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
inet 127.0.0.1/8 scope host lo  
    valid_lft forever preferred_lft forever  
inet6 ::1/128 scope host  
    valid_lft forever preferred_lft forever  
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP g  
roup default qlen 1000  
link/ether 00:0c:29:27:3c:b8 brd ff:ff:ff:ff:ff:ff  
inet 192.168.1.107/24 brd 192.168.1.255 scope global dynamic ens33  
    valid_lft 3447sec preferred_lft 3447sec  
inet6 fe80::719:2814:1e18:45a2/64 scope link  
    valid_lft forever preferred_lft forever  
ayman@ayman-virtual-machine:~$ ping -c 4 192.168.1.169  
PING 192.168.1.169 (192.168.1.169) 56(84) bytes of data.  
64 bytes from 192.168.1.169: icmp_seq=1 ttl=64 time=0.294 ms  
64 bytes from 192.168.1.169: icmp_seq=2 ttl=64 time=0.122 ms  
64 bytes from 192.168.1.169: icmp_seq=3 ttl=64 time=0.112 ms  
64 bytes from 192.168.1.169: icmp_seq=4 ttl=64 time=0.113 ms  
  
--- 192.168.1.169 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 2997ms  
rtt min/avg/max/mdev = 0.112/0.160/0.294/0.077 ms  
ayman@ayman-virtual-machine:~$
```

Ping réussi 192.168.1.169 – communication inter-VM sur le réseau interne ESXi

Conclusion

Le réseau interne d'ESXi permet la communication entre les VMs d'un même hôte sans passer par le réseau physique. Cette isolation réseau est typique des environnements de virtualisation en datacenter.

TP3 – Docker

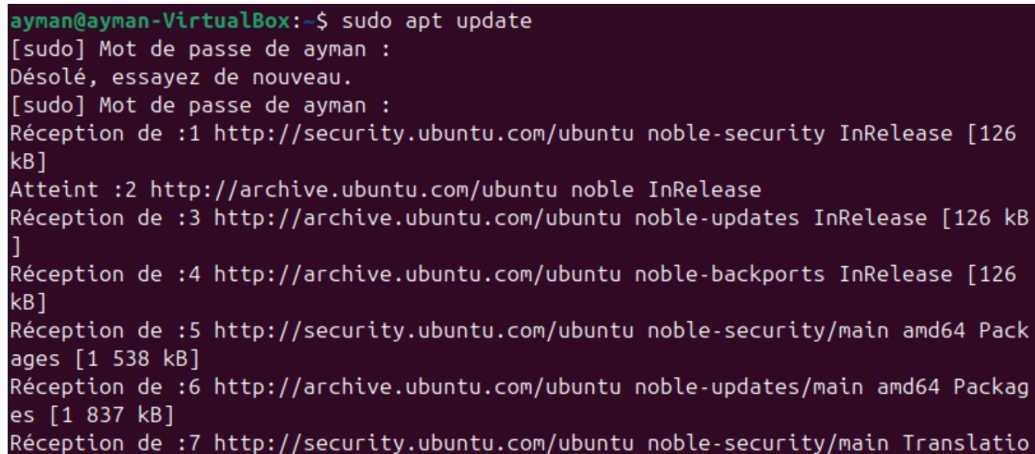
Objectif : Découvrir la conteneurisation avec Docker, comprendre la différence conteneurs/VMs, et déployer des applications multi-conteneurs avec Docker Compose.

1. Installation de Docker

Démarche

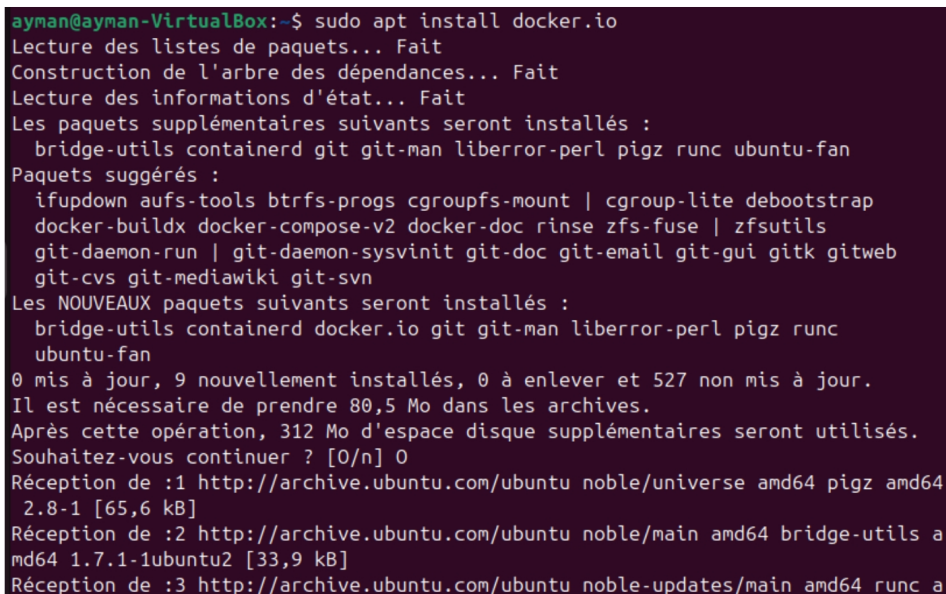
Mise à jour des paquets puis installation du paquet docker.io :

```
sudo apt update && sudo apt install docker.io
```



```
ayman@ayman-VirtualBox:~$ sudo apt update
[sudo] Mot de passe de ayman :
Désolé, essayez de nouveau.
[sudo] Mot de passe de ayman :
Réception de :1 http://security.ubuntu.com/ubuntu noble-security InRelease [126
kB]
Atteint :2 http://archive.ubuntu.com/ubuntu noble InRelease
Réception de :3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB
]
Réception de :4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126
kB]
Réception de :5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Pack
ages [1 538 kB]
Réception de :6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packag
es [1 837 kB]
Réception de :7 http://security.ubuntu.com/ubuntu noble-security/main Translatio
```

sudo apt update – mise à jour des dépôts avant installation Docker

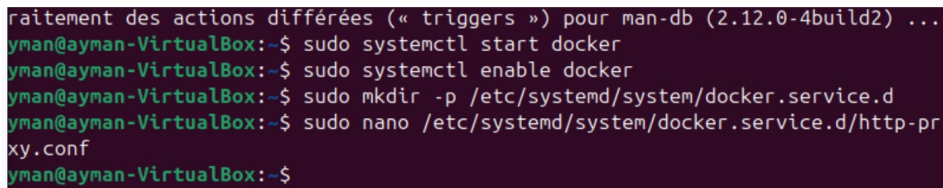


```
ayman@ayman-VirtualBox:~$ sudo apt install docker.io
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  bridge-utils containerd git git-man liberror-perl pigz runc ubuntu-fan
Paquets suggérés :
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap
  docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn
Les NOUVEAUX paquets suivants seront installés :
  bridge-utils containerd docker.io git git-man liberror-perl pigz runc
  ubuntu-fan
0 mis à jour, 9 nouvellement installés, 0 à enlever et 527 non mis à jour.
Il est nécessaire de prendre 80,5 Mo dans les archives.
Après cette opération, 312 Mo d'espace disque supplémentaires seront utilisés.
Souhaitez-vous continuer ? [O/n] O
Réception de :1 http://archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64
  2.8-1 [65,6 kB]
Réception de :2 http://archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils a
md64 1.7.1-1ubuntu2 [33,9 kB]
Réception de :3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 runc a
```

sudo apt install docker.io – installation de Docker

Démarrage du service et configuration du proxy IUT :

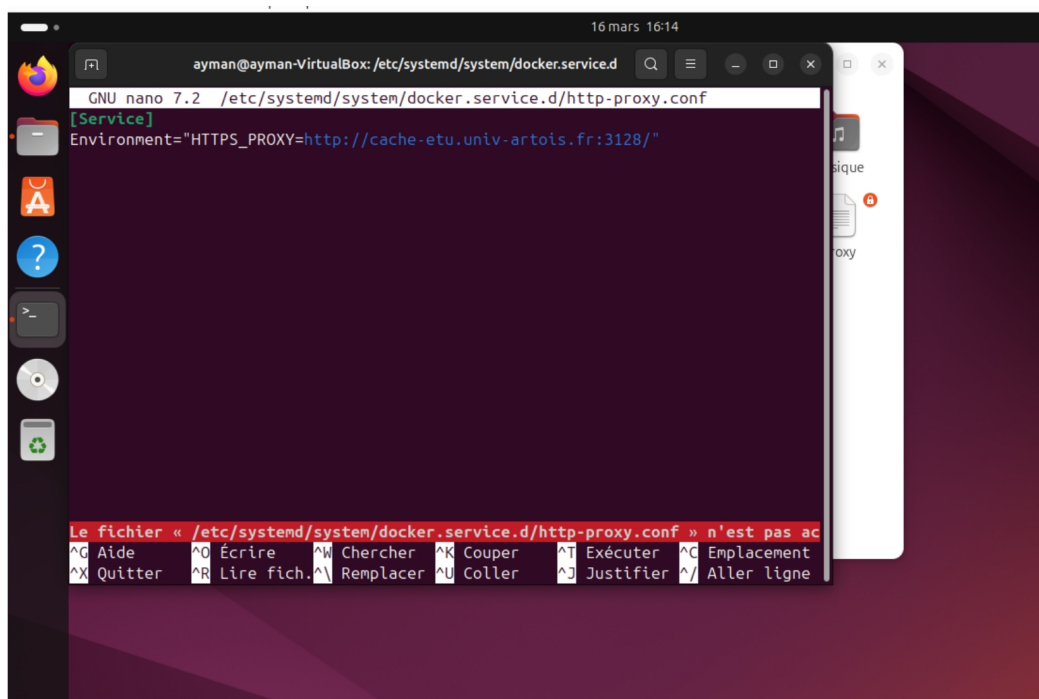
```
sudo systemctl start docker && sudo systemctl enable docker
```



```
raitement des actions différées (« triggers ») pour man-db (2.12.0-4build2) ...
yman@ayman-VirtualBox:~$ sudo systemctl start docker
yman@ayman-VirtualBox:~$ sudo systemctl enable docker
yman@ayman-VirtualBox:~$ sudo mkdir -p /etc/systemd/system/docker.service.d
yman@ayman-VirtualBox:~$ sudo nano /etc/systemd/system/docker.service.d/http-proxy.conf
yman@ayman-VirtualBox:~$
```

systemctl start/enable docker – démarrage et activation du service

Configuration du proxy pour permettre à Docker de télécharger les images :



Configuration du proxy dans /etc/systemd/system/docker.service.d/http-proxy.conf

Conclusion

Docker est disponible nativement dans les dépôts Ubuntu. La configuration du proxy est nécessaire à l'IUT pour permettre au daemon Docker de télécharger les images depuis le Docker Hub.

2. Utilisation de Docker

Premier container

Lancement du premier container – Docker télécharge l'image automatiquement :

```
docker run rtbethune/debian df -Th
```

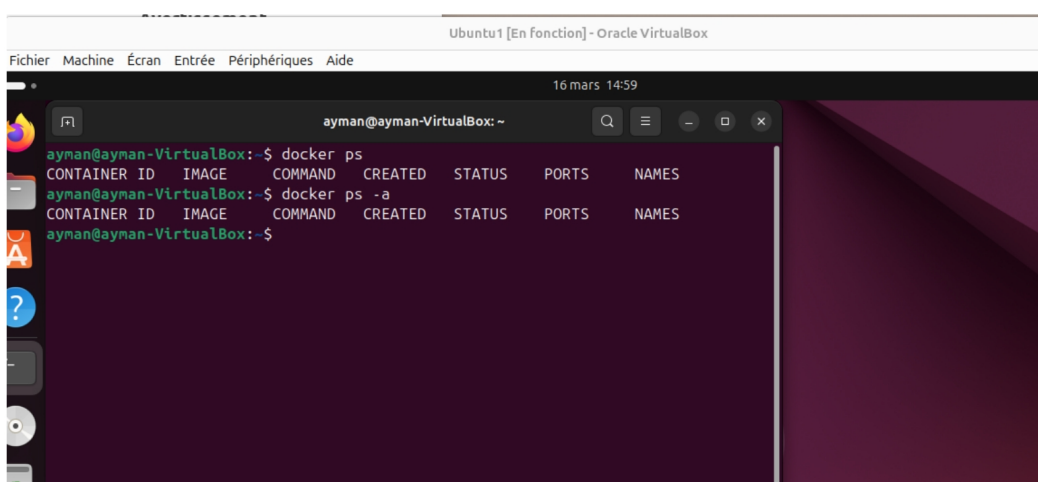
```
ayman@ayman-VirtualBox:/etc/systemd/system/docker.service.d$ systemctl daemon-reload
ayman@ayman-VirtualBox:/etc/systemd/system/docker.service.d$ systemctl restart docker
ayman@ayman-VirtualBox:/etc/systemd/system/docker.service.d$ docker run rtbethune/debian df -Th
Unable to find image 'rtbethune/debian:latest' locally
latest: Pulling from rtbethune/debian
df5590a8898b: Pull complete
f0556d053be8: Pull complete
Digest: sha256:09f58c01c1f8c74ecdcc013b2e2db34b957db9f43f1c597d29ff480660444e69
Status: Downloaded newer image for rtbethune/debian:latest
Filesystem      Type      Size  Used Avail Use% Mounted on
overlay          overlay   9.8G   8.2G   1.1G   89% /
tmpfs            tmpfs     64M    0    64M    0% /dev
shm              tmpfs     64M    0    64M    0% /dev/shm
/dev/sda2        ext4      9.8G   8.2G   1.1G   89% /etc/hosts
tmpfs            tmpfs     984M    0   984M    0% /proc/asound
tmpfs            tmpfs     984M    0   984M    0% /proc/acpi
tmpfs            tmpfs     984M    0   984M    0% /proc/scsi
tmpfs            tmpfs     984M    0   984M    0% /sys/firmware
ayman@ayman-VirtualBox:/etc/systemd/system/docker.service.d$ docker ps
```

docker run rtbethune/debian df -Th – téléchargement et exécution du container

Le container s'arrête dès que la commande est terminée. docker ps ne montre rien, docker ps -a montre le container stoppé :

```
docker ps docker ps -a
```

```
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
ayman@ayman-VirtualBox:/etc/systemd/system/docker.service.d$
```



docker ps (vide) et docker ps -a (container stoppé visible)

Container interactif Debian

Ouverture d'un terminal interactif dans le container :

```
docker run -ti --name=debian rtbethune/debian bash
```

```

ayman@ayman-VirtualBox:~$ docker run -ti --name=debian rtbethune/debian bash
root@c15307c9b9e5:/# cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 11 (bullseye)"
NAME="Debian GNU/Linux"
VERSION_ID="11"
VERSION="11 (bullseye)"
VERSION_CODENAME=bullseye
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
root@c15307c9b9e5:/# uname -a
Linux c15307c9b9e5 6.17.0-19-generic #19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Fri
Mar 6 23:08:46 UTC 2 x86_64 GNU/Linux
root@c15307c9b9e5:/# ping 172.31.19.88
bash: ping: command not found
root@c15307c9b9e5:/# cd ~
root@c15307c9b9e5:~# cd

```

Container Debian interactif – Debian 11 Bullseye, noyau Linux 6.17.0

Vérification de l'adresse IP du container et de la communication réseau :

```

ayman@ayman-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
    group default qlen 1000
    link/ether 08:00:27:00:03:88 brd ff:ff:ff:ff:ff:ff
    inet 172.31.19.88/20 brd 172.31.31.255 scope global dynamic noprefixroute en
    p0s3
        valid_lft 7775263sec preferred_lft 7775263sec
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN
    group default
    link/ether 92:17:8b:49:7d:56 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

```

COMMUNICATION AVEC LE SYSTEM HOTE :

ip a dans le container – interface docker0 172.17.0.0/16

```

bash: ./home/test.txt: Permission denied
ayman@ayman-VirtualBox:~$ ping 172.17.0.1
PING 172.17.0.1 (172.17.0.1) 56(84) bytes of data.
64 bytes from 172.17.0.1: icmp_seq=1 ttl=64 time=0.027 ms
64 bytes from 172.17.0.1: icmp_seq=2 ttl=64 time=0.024 ms
64 bytes from 172.17.0.1: icmp_seq=3 ttl=64 time=0.019 ms
64 bytes from 172.17.0.1: icmp_seq=4 ttl=64 time=0.024 ms
^C
--- 172.17.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3070ms
rtt min/avg/max/mdev = 0.019/0.023/0.027/0.003 ms
ayman@ayman-VirtualBox:~$

```

Ping 172.17.0.1 – communication réussie entre container et hôte

```

ayman@ayman-VirtualBox:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
ayman@ayman-VirtualBox:~$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
1855c9a66934   rtbethune/debian   "df -Th"                5 minutes ago    Exited (0) 5 minute
s ago         upbeat_shtern
ayman@ayman-VirtualBox:~$

```

docker ps -a – état des containers après expérimentation

Conclusion

Les containers Docker partagent le noyau de l'hôte mais sont isolés en réseau et système de fichiers. La version Debian du container est indépendante de l'OS hôte (Ubuntu). Un container s'arrête dès que son processus principal termine.

3. Docker Compose

Installation

```
apt install docker-compose
```



```

root@Ubuntu1:/home/user# docker ps
CONTAINER ID   IMAGE          COMMAND         CREATED        STATUS        PORTS
TS            NAMES
2a6bf530a019   rtbethune/debian "bash"         16 minutes ago Up 16 minutes
debian
root@Ubuntu1:/home/user#

```

Installation de docker-compose

Premier déploiement – serveur Apache

Création du fichier docker-compose.yml pour un serveur Apache :

```

version: '3.3' services:  web:      image: webdevops/php-apache:ubuntu-
16.04    ports:    - "80:80"    volumes:  - "$PWD/./app/"

```

```

root@Ubuntu1:/home/user# docker diff 2a6bf530a019
C /bin
A /bin/ip

```

docker-compose up -d – démarrage du serveur Apache

```

C /lib/x86_64-linux-gnu/security
A /lib/x86_64-linux-gnu/security/pam_cap.so
A /test

```

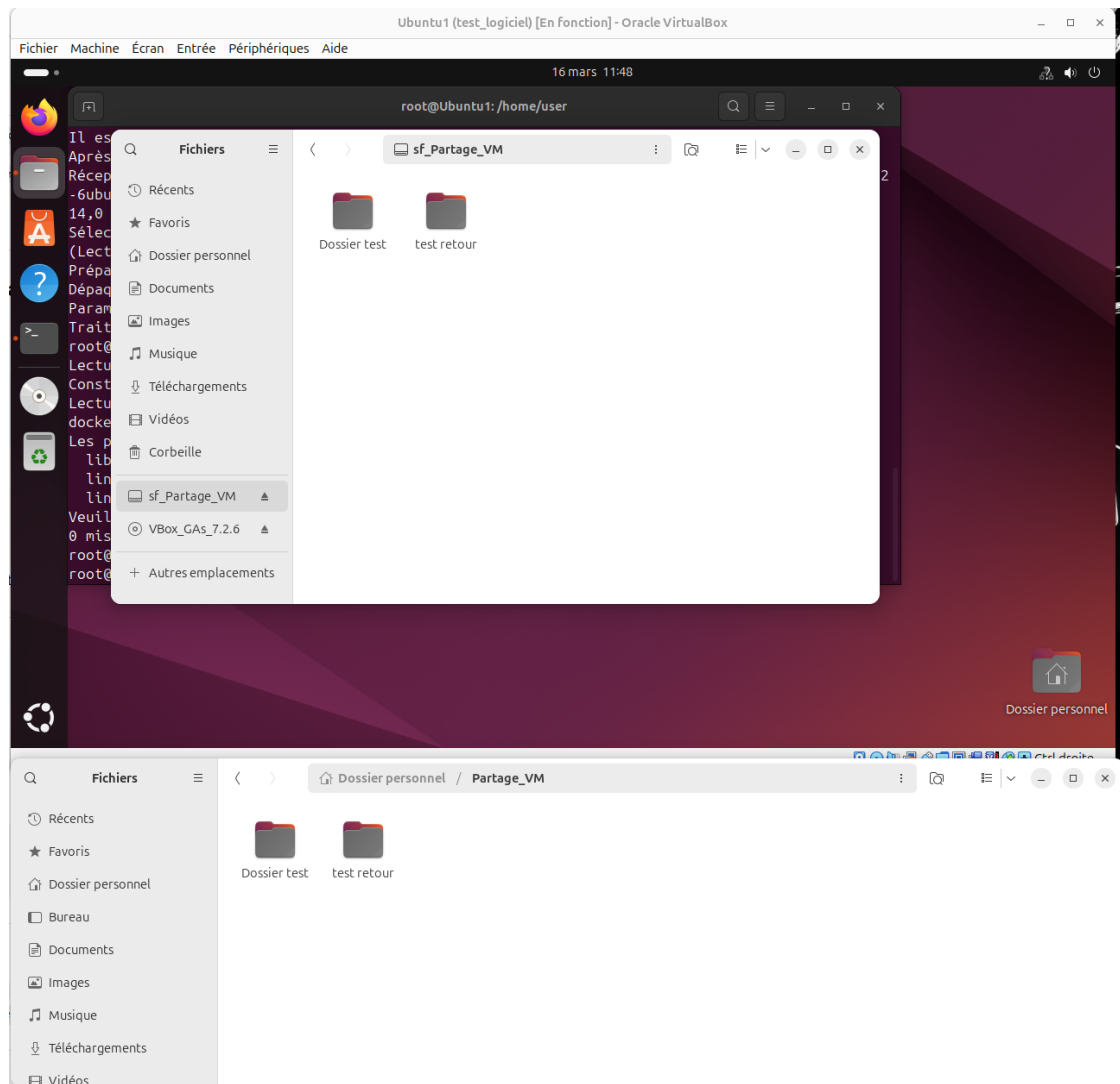
docker-compose --help – étude des paramètres

Déploiement WordPress + MySQL

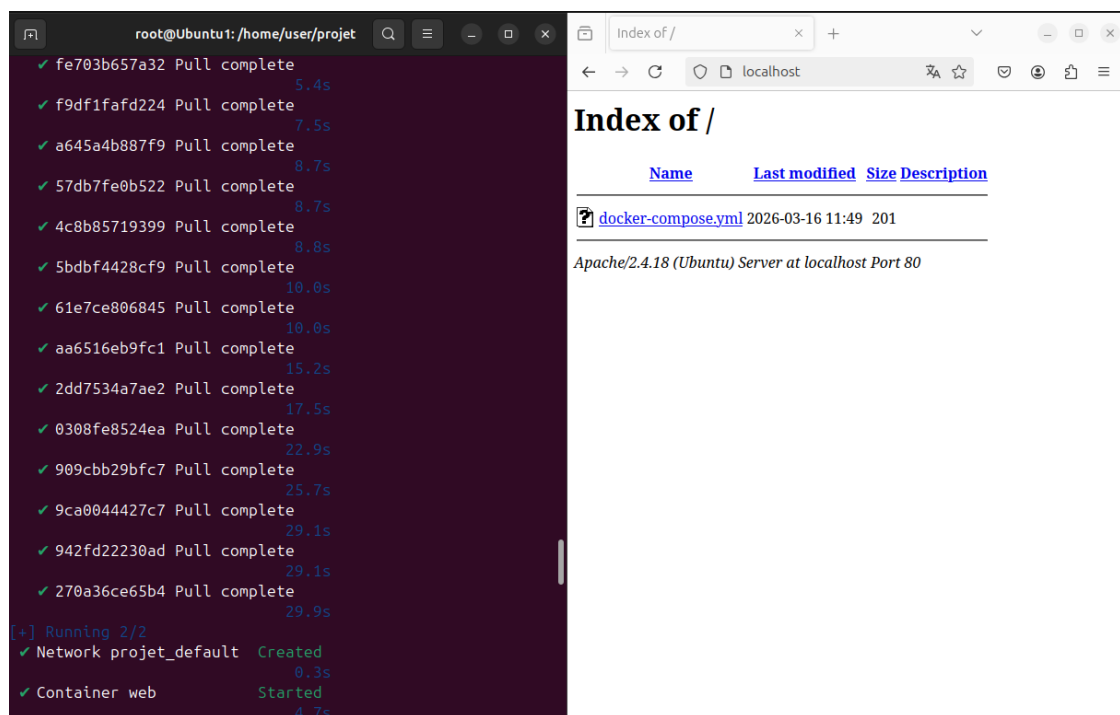
Configuration docker-compose.yml avec réseau dédié (10.16.0.0/24), volume persistant dbdata, MySQL et WordPress :

The image shows two terminal windows. The left window, titled 'samir@tp4-virtual-machine:~\$', displays the output of the 'ip a' command, showing network interfaces 'lo' and 'ens160'. The right window, titled 'j009-05-mb@j00905mb-virtual-machine:~\$', displays the output of the 'ip a' command, showing network interfaces 'lo' and 'ens160'. Both windows show ping statistics for the IP address 192.168.1.116.

docker-compose up – démarrage WordPress et MySQL



Interface WordPress accessible sur <http://localhost:8000>



Containers WordPress et MySQL actifs – vérification avec docker ps

4. Portainer – Interface graphique Docker

Installation

Ajout de Portainer au fichier docker-compose.yml :

```
portainer:  image: portainer/portainer  ports:    - "9000:9000"
volumes:   - /var/run/docker.sock:/var/run/docker.sock
```

```
GNU nano 7.2                                docker-compose.yml *
#Indique la version du format docker
version: '3.3'

#Le conteneur à lancer
services:

#nom du service
  web:

#Image du serveur apache 2 + php qui va tourner sur le conteneur
  image: webdevops/php-apache:ubuntu-16.04

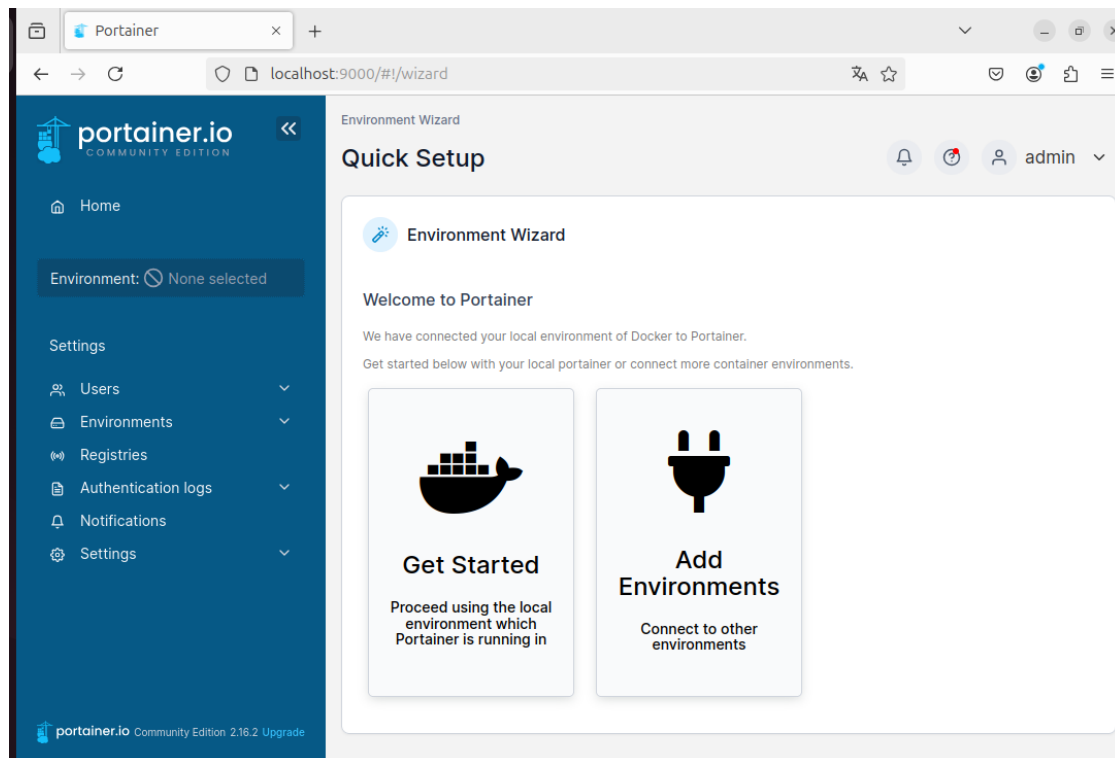
#Nom du conteneur
  container_name: web

#Ports exposés ( machine -> conteneur)
  ports:
    - "80:80"
    - "443:443"

#Monte le repertoire actuel de la machine dans le conteneur (/app/)
  volumes:
    - "$PWD:/app/"
```

Interface Portainer sur <http://localhost:9000> – dashboard de gestion

Portainer – liste des containers en cours d'exécution



Portainer – visualisation des réseaux Docker

5. Extensions – phpMyAdmin & PostgreSQL

Ajout de phpMyAdmin et PostgreSQL au fichier docker-compose.yml :

```
root@Ubuntu1:/home/user/projet# docker compose up -d
WARN[0000] /home/user/projet/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 15/15
✔ web Pulled 31.4s
  ✔ fe703b657a32 Pull complete 5.4s
  ✔ f9df1fafd224 Pull complete 7.5s
  ✔ a645a4b887f9 Pull complete 8.7s
  ✔ 57db7fe0b522 Pull complete 8.7s
  ✔ 4c8b85719399 Pull complete 8.8s
  ✔ 5bdbf4428cf9 Pull complete 10.0s
  ✔ 61e7ce806845 Pull complete 10.0s
  ✔ aa6516eb9fc1 Pull complete 15.2s
  ✔ 2dd7534a7ae2 Pull complete 17.5s
  ✔ 0308fe8524ea Pull complete 22.9s
  ✔ 909cbb29bfc7 Pull complete 25.7s
  ✔ 9ca0044427c7 Pull complete 29.1s
  ✔ 942fd22230ad Pull complete 29.1s
  ✔ 270a36ce65b4 Pull complete 29.9s
[+] Running 2/2
✔ Network projet_default Created 0.3s
✔ Container web Started 4.7s
```

phpMyAdmin ajouté – accessible et connecté à MySQL

```
root@Ubuntu1:/home/user/projet# docker compose up -d
WARN[0000] /home/user/projet/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 25/25
✔ wordpress Pulled 28.0s
[+] Running 0/1
  ✔ Network projet_mon_reseau Creating 0.2s
WARN[0029] Found orphan containers ([web]) for this project. If you removed or r[+] Running 4/4vice in you
r compose file, you can run this command with the --re ✔ Network projet_mon_reseau Created 0.3s
✔ Volume "projet_dbdata" Created 0.0s
✔ Container mysql Started 2.6s
✔ Container wordpress Started 2.0s
root@Ubuntu1:/home/user/projet#
```

PostgreSQL configuré et démarré via docker-compose

Authentification - Adminer x

localhost:8081

Langue: Français

Adminer 5.4.2

Authentification

Système	MySQL / MariaDB
Serveur	db
Utilisateur	
Mot de passe	
Base de données	

☒ Authentification ☐ Authentification permanente

Vue d'ensemble de tous les services dans Portainer

Conclusion

Docker Compose permet de déployer des stacks applicatives complexes en une seule commande. La séparation des services en containers indépendants favorise la modularité et la reproductibilité des environnements.

Synthèse comparative

Ces trois TP illustrent la complémentarité des technologies de virtualisation et de conteneurisation :

Critère	VirtualBox	VMware ESXi	Docker
Type	Hyperviseur type 2	Hyperviseur type 1	Conteneurisation
Isolation	Totale (OS complet)	Totale (OS complet)	Partielle (noyau partagé)
Performance	Moyenne	Élevée	Très élevée
Démarrage	Minutes	Minutes	Secondes
Usage	Dev / Formation	Production / Datacenter	Microservices / Dev

VirtualBox pour l'usage pédagogique, ESXi pour la production en datacenter, et Docker pour la conteneurisation légère. Ces trois outils répondent à des besoins complémentaires dans le cycle de vie des applications modernes.